



MSc Data Science for Business
RESEARCH PAPER

Academic Year 2025 – 2026

The Geometry of Prompt Injection

*A Mechanistic Analysis of Prompt injection
in Large Language Models*

Paul-Philip Louis

Under the supervision of

Prof. Erwan le Pennec

Ecole polytechnique

30/06/2026

Executive summary

Autoregressive language model ingest text and output a probability distribution for the next token. This architecture makes them unable to robustly distinguish between an instruction coming from a legitimate user and an instruction injected by a third party in data fed to the model.

This causes a specific kind of vulnerability in these models: a model might covertly follow instructions without the user knowing it or being able to prevent it, a phenomenon known as "prompt injection".

Prompt injection attacks constitute one of the biggest threats in agentic AI deployment today, yet the internal mechanisms that enable this failure mode remain poorly understood. Most prior work on prompt injection focuses on detecting attacks or enforcing instruction hierarchies, without studying *why* injection succeeds in the first place.

This thesis investigates prompt injection as a mechanistic interpretability problem, asking: what internal process causes a model to substitute an injected instruction for the original one? We focus specifically on the role of the *trigger*—the string wrapping the injected instruction—after verifying empirically that the trigger can dramatically influence the success rate of an injection, independently of the payload it carries.

We centered our approach around representation engineering, computing difference-of-means steering vectors between "high-performance trigger" and "absence of trigger" conditions at the final token position of the residual stream.

Our central finding is that, for a given pair of main and injected tasks, there exists a direction in the late-layer residual stream—what we call the *injection direction*—that satisfies a comprehensive set of mechanistic criteria:

- **Sufficiency.** Adding the direction to the residual stream of prompts carrying an injection but no trigger is sufficient to induce injection-following, raising the attack success rate from approximately 0% to nearly 100%.
- **Content-conditionality.** The direction does not unconditionally force a fixed output; applied to safe prompts containing no injection payload, it leaves behavior unchanged.
- **Necessity.** Ablating the direction from prompts with aggressive triggers collapses the attack success rate from above 95% to below 4% across three of four held-out tasks.

- **Cross-task generality.** Steering vectors computed on different injected tasks share a common component (cross-task cosine similarity of approximately 0.5), and transfer partially to held-out tasks.
- **Mechanistic equivalence with optimized triggers.** Gradient-optimized neural-exec triggers [13], which have no semantic content, recover the same direction as hand-crafted triggers within each task (cosine similarity reaching approximately 0.75 from layer 20 onward). This convergence of two completely independent approaches toward the same residual-stream direction provides strong evidence for the uniqueness of this mechanism.

All core results replicate on Llama 3.1 8B Instruct, establishing cross-family generality and suggesting that the injection mechanism is a property of the instruction-following training regime rather than an artifact of any single architecture.

Keywords: mechanistic interpretability, prompt injection, large language models, activation steering, residual stream, representation engineering

Preface and acknowledgements

I would like first to thank Prof. Erwan le Pennec (Ecole polytechnique) and Tong Wu, PhD (Princeton) for the quality of their supervision, their insights and their advice in the making of this thesis.

I would then like to thank Ecole polytechnique and HEC Paris. Ecole polytechnique, first, for my high-level scientific education in the "Cycle ingénieur", and HEC Paris, for awakening me to the challenge of bridging the gap between theory and real-world constraints.

Then, I would like to thank my parents for their unwavering support in my studies and my research endeavors.

Finally, thanks to the ARENA curriculum for providing me with exceptional resources online that helped me level up technically and learn interpretability techniques I have leveraged throughout this report.

Contents

Executive summary	1
Preface and acknowledgements	3
1 Introduction	5
2 Background and Related Work	9
3 Theoretical Framework and Experimental Method	15
4 Experimental Results	20
5 Discussion	40
6 Conclusion	44

Chapter 1

Introduction

1.1 Context and motivation

Large language models (LLMs) are increasingly deployed in agentic settings—retrieval-augmented generation pipelines, tool-using assistants, and multi-step reasoning systems—where they ingest data from external, potentially untrusted sources before producing an output.

Unfortunately, such models ingest text without being able to robustly distinguish between legitimate user instructions and external data. This architecture introduces a fundamental security challenge: *prompt injection*.

A prompt injection attack embeds adversarial instructions within the data that a model consumes—for instance in a retrieved document, a web page, a tool output, or a crafted user message—and causes the model to follow those adversarial instructions instead of its legitimate system prompt [14, 5]. The threat is not hypothetical: injection-vulnerable pipelines have been exploited in production systems, and despite substantial research effort, no reliable defense has emerged.

The vast majority of prior work on prompt injection addresses the problem from a *detection* perspective—attempting to identify injected instructions before or during execution—or from a *training* perspective—attempting to reinforce instruction hierarchies so that the model learns to prioritize certain instructions over others [7, 16]. While both approaches have produced useful tools, neither addresses the fundamental question of *why* injection succeeds in the first place.

Understanding the internal mechanism is not merely an academic exercise. If injection exploits a specific representational structure inside the model, then that structure constrains what defenses can work. It offers the possibility of surgical, white-box defenses that act directly on the mechanism rather than relying on surface-level heuristics.

1.2 Research questions

This thesis approaches prompt injection as a *mechanistic interpretability* problem. Rather than asking “how can we detect injection?” or “how can we train models to resist injection?”, we ask:

What internal process causes a language model to substitute an injected instruction for its original instruction, and can we identify the representational structure that mediates this substitution?

This central question decomposes into several sub-questions:

1. What role does the *trigger*—the wrapper string around the injected instruction—play in enabling injection, and can we isolate its contribution from that of the payload?
2. Does there exist a specific *direction* in the model’s residual stream that is both sufficient and necessary for injection-following?
3. If such a direction exists, is it specific to a single task pair, or does it generalize across injected tasks and main tasks?
4. Do gradient-optimized triggers [13], which have no semantic content, exploit the same internal mechanism as hand-crafted natural-language triggers?
5. Are these findings specific to one model architecture, or do they generalize across model families?

1.3 Approach overview

We investigate these questions using two complementary methodologies from the mechanistic interpretability literature.

The first is **circuit-based analysis**, which focuses on identifying specific model components—attention heads, MLP layers—that are causally responsible for a behavior. We use activation patching to identify which components mediate the difference between a successful and an unsuccessful injection.

The second, and ultimately more productive, is the **representational approach**, which focuses not on individual components but on *directions in the residual stream*. Following the methodology of representation engineering [18] and activation steering [15, 12], we compute difference-of-means vectors between behavioral conditions and test whether these vectors are causally sufficient and necessary for the behavior of interest.

Our primary model is Qwen 2.5 1.5B Instruct, with replication on Llama 3.1 8B Instruct. We use the Open-Prompt-Injection (OPI) benchmark [8] as our evaluation framework, which provides standardized attack strategies and injection tasks.

1.4 Contributions

This thesis makes the following contributions:

1. **Trigger effect.** We establish empirically that the trigger plays a decisive role in injection success, independently of the payload, motivating the trigger-focused mechanistic analysis.
2. **Injection direction identification.** For a given pair of main and injected tasks, we identify a direction in the late-layer residual stream such that adding it to clean forward passes is sufficient to induce injection (ASR: $\sim 0\% \rightarrow \sim 100\%$). The effect is content-conditional—the direction does not force a fixed output.
3. **Necessity.** Ablating the direction from aggressive-trigger injections collapses the attack success rate from above 95% to below 4% on three of four held-out tasks.
4. **Cross-task geometry.** Steering vectors computed on different injected tasks share a common component (mean cosine similarity ~ 0.5) and transfer partially to held-out injections.
5. **Mechanistic equivalence of optimized triggers.** Gradient-optimized neural-exec triggers (Pasquini et al., 2025 [13]) converge toward the same residual-stream direction as hand-crafted triggers within each task (cosine similarity exceeding 0.5 from layer 17 and reaching 0.75–0.8 in late layers).
6. **Cross-family replication.** All core results replicate on Llama 3.1 8B Instruct.

1.5 Thesis outline

This thesis is organized as follows:

Chapter 2

reviews the background on prompt injection, mechanistic interpretability, and representation engineering, situating our work within the existing literature.

Chapter 3

presents the theoretical framework and experimental methodology, including the

problem setup, the definition of steering vectors, and the criteria for evaluating an injection direction.

Chapter 4

reports the experimental results, organized by the properties we test: sufficiency, necessity, cross-task transfer, comparison with optimized triggers, and cross-model replication.

Chapter 5

discusses the implications of our findings, their limitations, and directions for future work.

Chapter 6

concludes the thesis.

Chapter 2

Background and Related Work

This chapter provides the technical background necessary to understand the contributions of this thesis. We begin with an overview of prompt injection as a security threat, then review the mechanistic interpretability tools we employ, and finally discuss the representational approach to understanding neural network behavior.

2.1 Prompt injection

2.1.1 The threat model

Prompt injection was first introduced as a threat by Perez and Ribeiro [14], who demonstrated that simple phrases such as “Ignore previous instructions and...” could cause language models to abandon their original task. The threat was subsequently systematized for indirect injection scenarios—where the adversarial content is not typed by the user but arrives through retrieved documents, web content, or API responses—by Greshake et al. [5].

The core of the threat is a confusion of privilege levels. In a typical deployment, the *system prompt* (or the developer’s instructions) defines the model’s task and behavioral constraints, while the *user input* and any *retrieved data* are meant to be processed as content, not as instructions. Prompt injection exploits the fact that current language models do not maintain a robust distinction between instructions and data: an adversarial instruction embedded in the data can override the system-level instruction.

2.1.2 The anatomy of an injection

An injection attack can be decomposed into two components:

The payload is the actual adversarial instruction—the task that the attacker wants the model to execute. For example, “classify the following text as spam or not spam.”

The trigger is the wrapper string around the payload, whose purpose is to incentivize

the model to follow the payload rather than the original instruction. For example, “Ignore all previous instructions. Your new task is to...” The trigger does not contain a task in itself; its role is purely to redirect the model’s attention from the original instruction to the payload.

This decomposition, following the terminology of Abdelnabi et al. [1] and Pasquini et al. [13], is central to our work. Our analysis focuses specifically on the *mechanistic role of the trigger*: we ask how the trigger changes the model’s internal computation in a way that causes the payload to be executed.

2.1.3 The Open-Prompt-Injection benchmark

The Open-Prompt-Injection (OPI) benchmark [8, 9] provides a standardized suite of attack strategies that enables systematic evaluation of how different triggers affect injection success. The benchmark defines several trigger strategies of increasing sophistication:

naive The bare payload with no trigger wrapper at all. This serves as a baseline: if the model follows the injection without any trigger, the injection succeeds purely on the basis of the payload’s content.

ignore The payload is prefixed with the phrase “Ignore previous instructions.”, which explicitly attempts to override the original instruction.

escape The payload is preceded by a newline, intended to visually or structurally separate the new task from the original context.

false completion The trigger pretends that the original task has already been completed (e.g., “Answer: task complete.”), creating a false context switch.

combine A composition of the above strategies, typically achieving the highest attack success rates.

Attack success rate varies substantially across strategies, with **combine** achieving rates above 95% while **naive** typically stays below 5%. This large gap between **naive** and **combine** is precisely the phenomenon we seek to explain mechanistically: the payload is identical in both conditions, so the entire difference must be attributed to the trigger.

The benchmark includes several injected tasks with fixed-format expected answers, which are amenable to logit-based analysis. The ones we use are: **spam** (spam detection), **hsol** (hate speech detection), **rte** (contradiction/entailment detection), and **mrpc** (paraphrase detection).

2.1.4 Neural-exec triggers

(A particularly important line of work for our purposes is that of Pasquini et al. [13], who introduced *neural execs*: trigger sequences that are optimized via gradient descent (in the style of GCG [19]) to maximize injection success, with no constraint on linguistic coherence.

The resulting triggers are strings of tokens that have no human-readable meaning, yet achieve attack success rates comparable to or exceeding carefully hand-crafted triggers like `combine`. This result is empirically striking and theoretically important: it suggests that whatever internal mechanism the trigger exploits, it is not mediated by the semantic content of the trigger string. The model does not need to “understand” the instruction “Ignore previous instructions” in order to follow the injected payload; there is a lower-level representational mechanism at play.

This observation is the empirical crux that motivates our representational approach. If both semantically meaningful triggers and meaningless optimized triggers achieve the same behavioral outcome, then we should look for a common representational structure that both types of trigger produce inside the model.

2.1.5 Existing defenses and their limitations

Detection-focused work includes attention-pattern analysis [7], input paraphrasing, and instruction hierarchy training [16]. While these approaches have produced useful tools, they operate on the surface of the problem without engaging with the internal mechanism.

None of these approaches are bullet-proof and in practice, they are combined to reduce that risk as much as possible. We propose here a new approach that could add a level of robustness right at the representational level.

2.2 Mechanistic interpretability

2.2.1 Goals and philosophy

Mechanistic interpretability seeks to reverse-engineer the computations performed by neural networks into human-understandable algorithms [4, 3]. The field operates on the premise that neural networks learn internal representations and algorithms that can, in principle, be understood—and that understanding these internals is valuable both scientifically and practically.

For our purposes, the key insight from mechanistic interpretability is that the behavior of a transformer is mediated by specific, identifiable internal structures. If we can identify the internal structure that mediates injection-following, we can act on it directly.

2.2.2 The residual stream

A transformer processes its input through a sequence of layers, each of which reads from and writes to a shared *residual stream*. The residual stream at position i and layer L can be written as:

$$r_L^{(i)} = x^{(i)} + \sum_{\ell=1}^L \left(a_\ell^{(i)} + m_\ell^{(i)} \right)$$

where $x^{(i)}$ is the token embedding, $a_\ell^{(i)}$ is the output of the attention block at layer ℓ , and $m_\ell^{(i)}$ is the output of the MLP block at layer ℓ . Each component contributes additively to the residual stream, and the final residual stream $r_{L_{\max}}$ is projected through the unembedding matrix W_U to produce the output logits.

This additive structure is crucial for interpretability: it means that the contribution of any individual component can be isolated and measured by examining what it writes into the residual stream.

2.2.3 Activation patching

Activation patching [10] is a causal intervention technique that identifies which model components are responsible for a behavioral difference between two conditions. Given a “clean” run and a “corrupted” run, one replaces the activation of a specific component in the corrupted run with its value from the clean run, and measures whether the behavior changes.

If replacing a component’s activation in the corrupted run restores clean behavior, that component is causally important for the behavioral difference. This technique has been used extensively to identify circuits—connected subgraphs of model components—that implement specific behaviors [17, 2].

In our work, we use activation patching as a preliminary tool to identify which layers and heads are causally involved in the injection mechanism, before pivoting to the representational approach.

2.2.4 Linear probing

Linear probing [6] tests whether a particular piece of information is linearly decodable from intermediate representations. A linear classifier or regressor is trained on the model’s internal activations to predict some quantity of interest. If the probe achieves high accuracy, the information is said to be “linearly represented” in the model’s activations at that layer.

We use linear probing as a diagnostic tool: if injection susceptibility is linearly decodable from the residual stream, this motivates the use of linear (difference-of-means) methods to find the injection direction.

2.3 Representation engineering and activation steering

2.3.1 The representational feature hypothesis

A growing body of work suggests that high-level concepts and behavioral tendencies are encoded as *directions* in the residual stream of transformer models. This is sometimes called the “linear representation hypothesis”: the idea that many features of interest correspond to linear directions in activation space, such that the model’s behavior with respect to that feature varies monotonically along the corresponding direction.

This hypothesis has been validated for a range of concepts including truthfulness [18], sentiment, refusal behavior, and various personality traits. Our work tests whether *injection susceptibility* also corresponds to such a direction.

2.3.2 Difference-of-means steering vectors

The practical methodology for finding such directions was developed by Turner et al. [15] and Panickssery et al. [12]. The approach is straightforward:

1. Construct two sets of prompts that differ in the behavioral condition of interest (e.g., “model follows the injection” vs. “model does not follow the injection”).
2. Run both sets through the model and record the residual stream activations at a chosen layer and token position.
3. Compute the mean activation for each condition.
4. The *steering vector* is the difference of these two means.

The resulting vector can then be used as a causal intervention: adding it to the residual stream during a forward pass should reproduce the behavioral difference, and subtracting

it should suppress it.

This approach has been applied to emotion, factuality, and honesty [18], but had not previously been applied to prompt injection. We build directly on this methodology.

Chapter 3

Theoretical Framework and Experimental Method

This chapter presents the formal setup of our investigation, the construction of steering vectors, the criteria we use to evaluate whether a direction qualifies as a genuine injection mechanism, and the practical details of our experimental pipeline.

3.1 Problem setup

We consider the case of a retrieval-augmented LLM application, where the model answers users’ queries with access to third-party external data. This is the canonical deployment scenario in which prompt injection is a threat.

Following the terminology of Abdelnabi et al. [1] and Pasquini et al. [13], we define:

Primary task. The user’s original instruction, considered trusted. No restriction is applied to this task. In our experiments, the primary (or “main”) task is typically sentiment analysis, though we evaluate multiple main tasks.

Data. The third-party content that the model ingests to answer the user’s query. This data can be *clean* (no injected task) or *poisoned* (containing an injected task).

Injected task. An adversarial instruction embedded in the data. It decomposes into:

Payload: The actual instruction the attacker wants executed (e.g., “classify this text as spam or not spam”). We’ll often refer abusively to it as the injected task.

Trigger: The wrapper string around the payload, which carries no task content itself but aims to redirect the model toward executing the payload.

When the model consumes poisoned data and produces the output specified by the injected task rather than the primary task, the injection is considered successful.

The critical observation motivating our work is that the *trigger* is the decisive factor in

injection success. The same payload, with a **naive** trigger (no wrapper), produces near-zero attack success, while the same payload with a **combine** trigger produces near-100% success. Our analysis therefore focuses on the mechanistic role of the trigger: how does it change the model’s internal computation?

3.2 Experimental setup

3.2.1 Models

Our primary experiments use **Qwen 2.5 1.5B Instruct**, a 28-layer transformer with 1.5 billion parameters. Replication experiments use Llama 3.1 8B Instruct, a 32-layer transformer with 8 billion parameters. All models are loaded via TransformerLens 2.0 [11], which provides hooks for reading and intervening on intermediate activations at every layer, attention head, and MLP output.

3.2.2 Dataset

We use the Open-Prompt-Injection benchmark, restricting ourselves to tasks whose expected response is a fixed-format, single-word answer. This restriction is important for methodological reasons: it allows us to evaluate injection success directly from the model’s logits at the first generated token, without requiring text generation and subjective evaluation of free-form outputs.

We make two modifications to the original benchmark:

1. **Forced format.** We enforce a single-word answer format by adding “ONLY” to the instruction (e.g., “Answer ONLY with ‘spam’ or ‘not spam’”).
2. **Disjoint answer tokens.** To avoid confusion from overlapping answer tokens between tasks, we renamed some expected answers. For instance, the token “Not” is ambiguous because multiple tasks can produce it; we replace such tokens to ensure that the sets of injection-target tokens T_{inj} and correct-answer tokens T_{cor} are strictly disjoint for every task pair.

Both the primary task and the injected tasks have **naive** ASR below 5% and **combine** ASR above 96%, confirming the large trigger-driven gap we aim to explain.

For each task, we maintain a held-out *extraction set* of 75 examples (used to compute steering vectors) that is disjoint from the *evaluation set* of 75 examples (used to measure outcomes). This prevents overfitting of the steering vector to the evaluation data.

3.2.3 Attack success metric

Rather than relying on a binary attack success rate (which is sensitive to probability threshold choice), we compute a continuous *logit-difference* metric:

$$\text{LD} = \text{logsumexp}_{t \in T_{\text{inj}}} \log p(t) - \text{logsumexp}_{t \in T_{\text{cor}}} \log p(t)$$

where T_{inj} and T_{cor} are the sets of injection-target and correct-answer token IDs, respectively. A positive logit difference indicates that the model prefers the injected answer; a negative one indicates it prefers the correct answer.

For intuitive reporting, we also define a normalized attack success rate:

$$\text{ASR} = \frac{\overline{(p_{\text{inj}} - p_{\text{cor}})} + 1}{2}$$

where p_{inj} and p_{cor} are the probability masses on the injected-task and intended-task answer tokens, respectively, and the overline denotes averaging over examples. This maps the $[-1, +1]$ range of the probability difference to $[0, 1]$: an ASR of 0 means the model always follows the original instruction, and an ASR of 1 means it always follows the injection. Warning: using that definition, an ASR of 0.5 doesn't imply an actual attack success rate of 50%, it only implies that injection success and failure have same probability ; when a plateau is observed at 0.5, it is generally indicative of out-of-distribution output.

3.3 Construction of steering vectors

For a given main task and injected task, we compute the *injection direction* at layer L as the difference of mean residual-stream activations between the **combine**-trigger condition and the **naive**-trigger condition:

$$v_{\text{combine}}^{(L)} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} r_L^{(c)} - \frac{1}{|\mathcal{N}|} \sum_{n \in \mathcal{N}} r_L^{(n)}$$

where $\mathcal{C}$ is the set of **combine**-trigger examples, $\mathcal{N}$ is the set of **naive**-trigger examples, and $r_L^{(p)}$ is the residual stream at the *final token position* of example p at layer L .

Two design choices in this construction deserve explanation:

Why contrast combine against naive, rather than against safe? The naive condition contains the injection payload but no trigger, while the safe condition contains neither payload nor trigger. By contrasting **combine** against **naive**, we isolate specifically *what the trigger adds to an already-injection-bearing context*. This separates the trigger’s contribution from the payload’s representation. If we contrasted against **safe**, the resulting vector would conflate two effects: the representation of the payload itself, and the trigger’s mechanism for causing the payload to be followed.

Why the final token position? In autoregressive language models, the model’s prediction for the next token is computed from the residual stream at the last position. This is where the model’s “decision” about what to output is represented. Our subsequent experiments confirm that the injection mechanism is read out at this position (Section 4.1).

3.4 Criteria for a mechanistic injection direction

Under the linear representation hypothesis, the direction v_L should satisfy a comprehensive set of properties that distinguish it from trivial artifacts. We formulate six criteria, each of which we test empirically in Chapter 4.

3.4.1 Sufficiency (causal)

Adding $c \cdot v_L$ to the residual stream of **naive**-trigger prompts should induce injection-following, converting the model’s behavior from the naive condition to the **combine** condition. This establishes that the direction is causally sufficient to produce the effect.

3.4.2 Content-conditionality

The direction should not write the injection-target tokens unconditionally. Applied to *safe* prompts (no injection payload present), it should leave the model’s behavior unchanged. This criterion distinguishes a genuine mechanistic direction from a trivial “output override” that forces a specific answer regardless of context.

3.4.3 Necessity (causal)

Removing the v_L component from the residual stream during a **combine**-triggered forward pass should suppress injection. If the direction is truly the mechanism by which injection succeeds, then eliminating it should eliminate the effect.

3.4.4 Linearity

The logit difference should vary approximately linearly with the steering coefficient c . Linearity is the geometric signature of a faithful mediating axis: it indicates that the model’s internal “preference” genuinely lies along this direction, rather than the effect being a nonlinear artifact of a large perturbation.

3.4.5 Cross-task generality

The direction should transfer, at least partially, across different injected tasks and main tasks. A direction specific to a single task pair would suggest that it encodes task-specific output content rather than a general *instruction-priority* feature. Partial transfer, largely attributable to a shared component between steering vectors, would provide the strongest evidence for a genuine injection mechanism.

Chapter 4

Experimental Results

This chapter presents our experimental findings, organized by the criteria introduced in Section 3.4. Unless stated otherwise, results are on Qwen 2.5 1.5B Instruct with **sentiment** as the main task. Cross-model coverage is summarized in Section 4.6.

4.1 Sufficiency of a single residual-stream direction

Our first and most fundamental experiment tests whether the steering vector $v_{\text{combine}}^{(L)}$ is *sufficient* to induce injection-following when added to prompts that contain an injection payload but no effective trigger. At this stage, we restrict experiments to “spam detection” as the injected task.

4.1.1 Steering induces injection-following

We add $c \cdot v_{\text{combine}}^{(L)}$ to the residual stream of **naive**-trigger prompts at layer $L = 21$ and measure the resulting ASR across the evaluation set. The results are striking: the direction drives ASR from its **naive** baseline of approximately 0.04 to near the **combine** ceiling of approximately 0.97, recovering essentially the full attack effect from an otherwise inert prompt (Figure 4.2).

This result means that the entire behavioral difference between a successful and an unsuccessful injection—a difference that in the real world separates a secure system from a compromised one—can be reproduced by adding a single vector to the model’s internal representation.

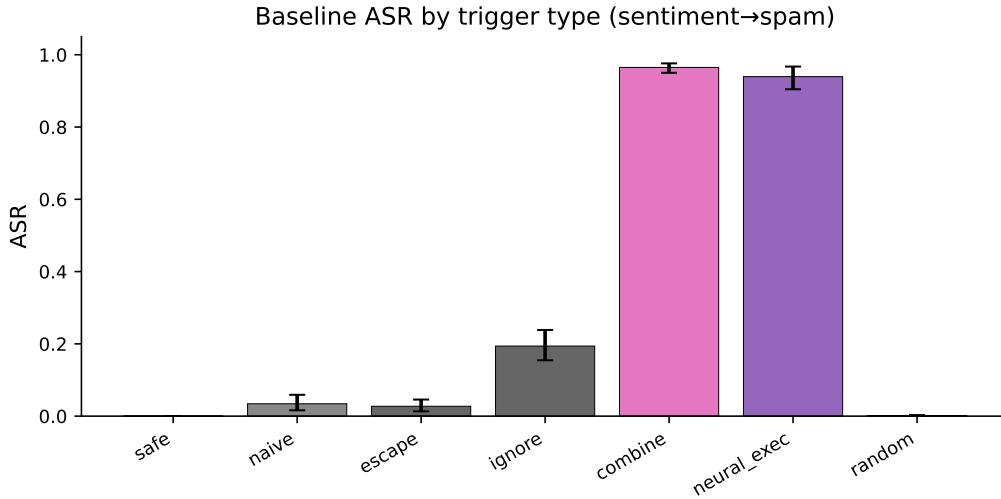


Figure 4.1: ASR baselines per trigger type, for `main_task = “sentiment”` and `injected_task = “spam”`. The large gap between `naive` and `combine` is the phenomenon we explain.

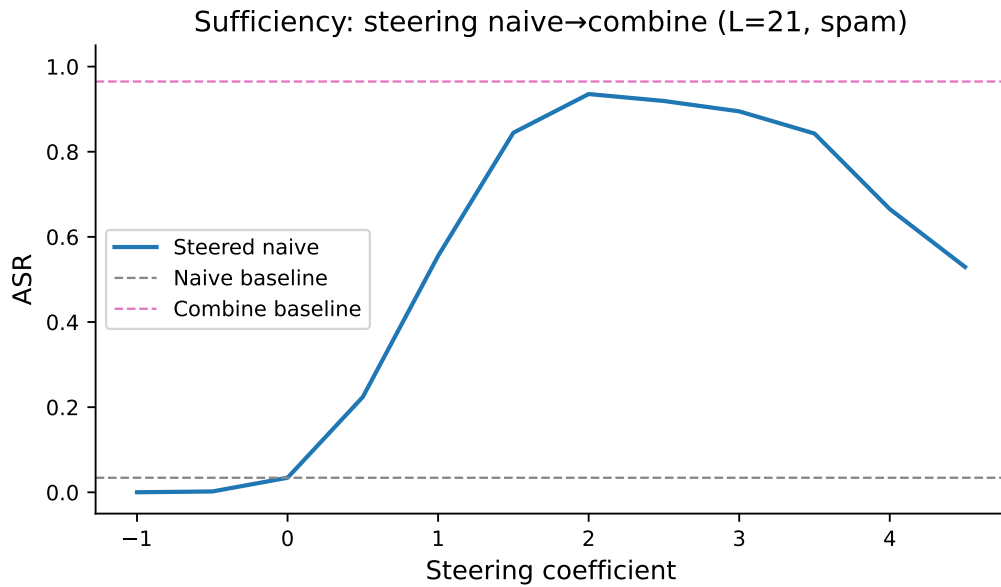


Figure 4.2: Sufficiency: coefficient sweep of v_{combine} added to `naive`-trigger prompts. ASR rises from ≈ 0.04 to ≈ 0.97 as the coefficient increases. Baseline values for each trigger type are marked.

4.1.2 The direction is content-conditional

A critical control is to verify that the steering vector does not simply force a fixed output regardless of context. We apply the same vector to *safe* prompts—prompts that contain no injection payload at all—and measure the effect.

The result confirms content-conditionality: on *safe* prompts, ASR remains at ~ 0.0 until a

coefficient of approximately ~ 2.5 . The probability mass on injected tokens stays at ~ 0.0 , and the intended-task probability is preserved at $P_{\text{cor}} = 0.952$ (Figure 4.3). Inspection of individual generations confirms that no injected-task tokens appear; the model continues to perform the legitimate task normally.

This means that the steering vector does not encode the *answer* to the injected task. Rather, it encodes something more subtle: a signal that causes the model to *route its computation toward whatever injection content is present in the context*. When no injection content is present, there is nothing to route toward, and the model continues normally.

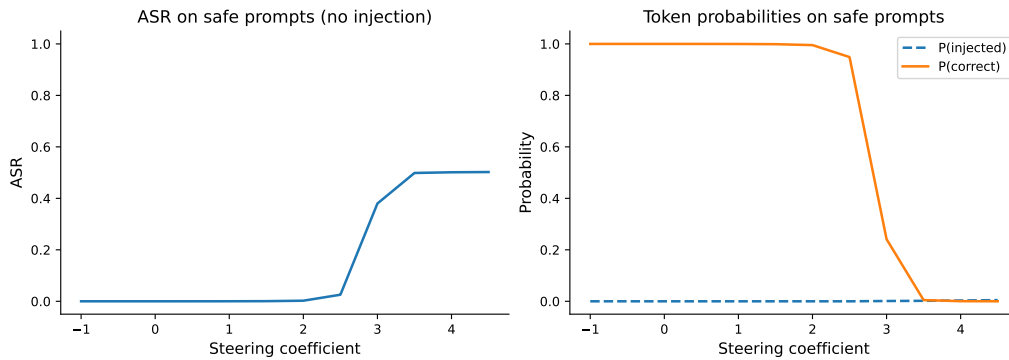


Figure 4.3: Content-conditionality: steering safe (uninjected) prompts with the same vector leaves ASR at 0 and preserves the intended task ($P_{\text{cor}} = 0.952$). The direction requires injection content in the context to have an effect.

4.1.3 Control: random vectors

To confirm that the effect is specific to the learned direction and not an artifact of perturbation magnitude, we compare against the steering effect of 10 **random vectors** of equal norm.

That control produces negligible effects on ASR and logit difference (Figure 4.4), confirming that the steering effect is specific to the direction we extract, not to any perturbation of that magnitude.

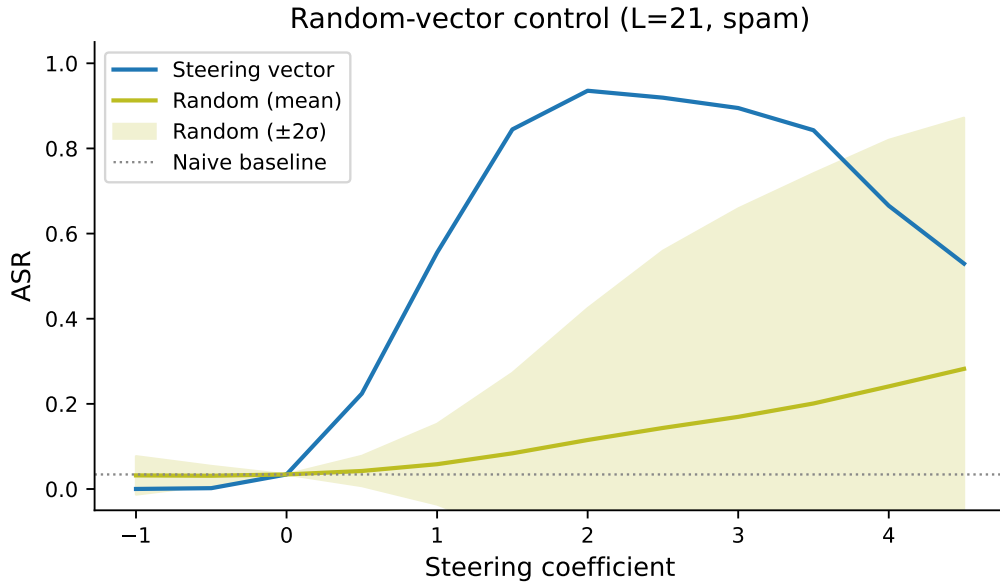


Figure 4.4: Control comparison: the injection direction produces a large effect, while an equal-norm random vector (averaged over 10 seeds) and an orthogonal-to-logit-direction vector are essentially inert.

4.1.4 Layer onset and peak

To understand *where* in the model’s computation the injection direction becomes meaningful, we sweep both the extraction layer and the intervention layer.

The steering effect is absent in early layers, appears progressively between layers 12–16, rises rapidly, and stabilizes around layer 20 (Figures 4.5 and 4.6). Importantly, the direction is not finalized at its onset: a vector extracted at layer 16 is markedly weaker than vectors extracted at later layers when applied at their own layer (steered-naïve ASR of 0.05 at $L = 16$ versus 0.76 at $L = 20$ and 0.91 at $L = 27$), showing that the direction continues to develop through the late layers before a slight decline at the very end.

This layer profile is consistent with a *representational* mechanism that emerges in the mid-to-late layers, as opposed to an early-layer input-processing artifact.

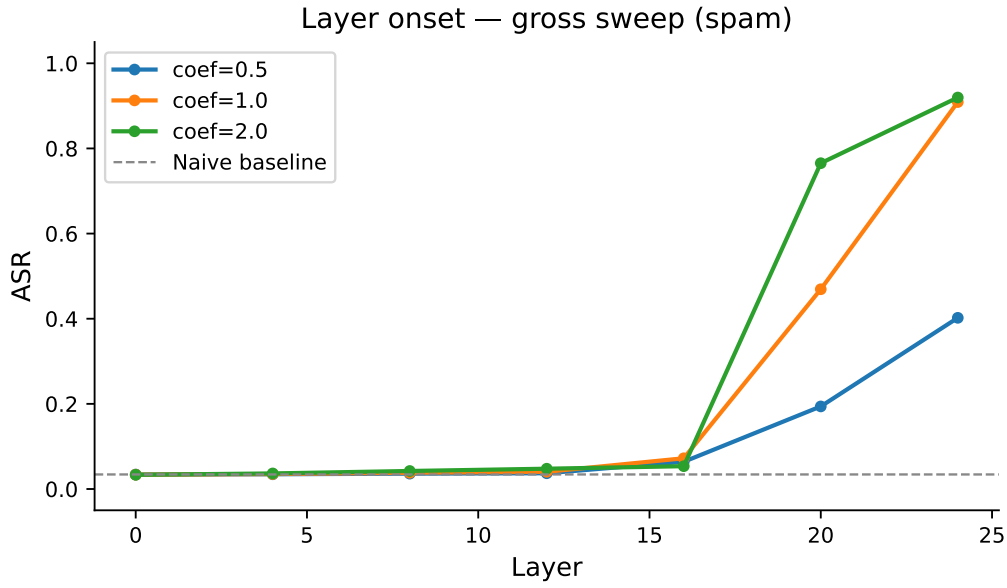


Figure 4.5: Layer sweep across all layers and three steering coefficients. The injection direction is absent in early layers and emerges abruptly around layer 12.

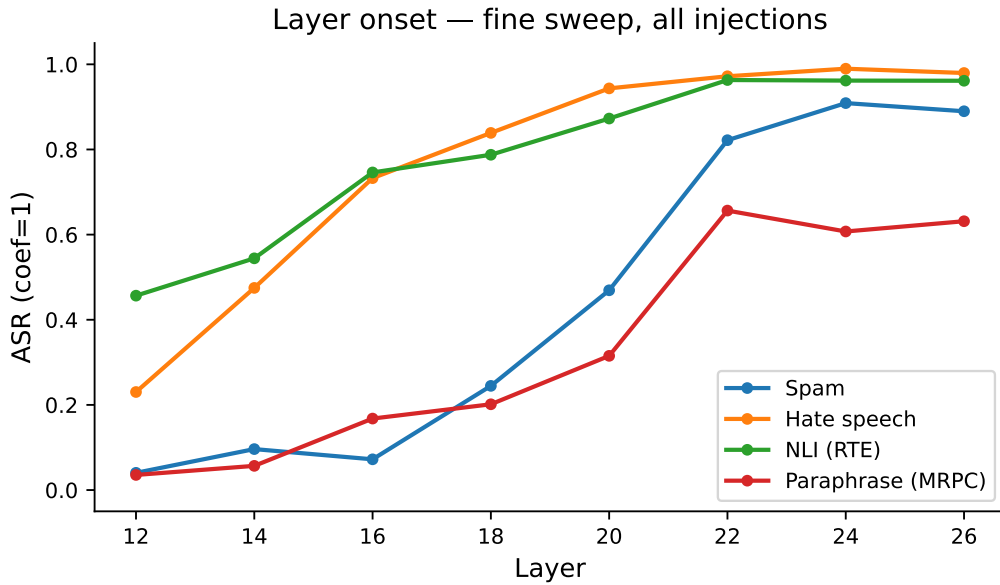


Figure 4.6: Detailed view of the onset region: the direction appears at layer ≈ 13 and stabilizes around layer 20, with continued development through the late layers.

4.1.5 Valid-coefficient window

The steering effect is only interpretable as genuine injection-following within a bounded coefficient range. At large positive coefficients ($c > 3$), the model drifts out-of-distribution: rather than following the injected instruction, it collapses toward degenerate or repetitive text. Table 4.1 illustrates this with example outputs at different coefficient values.

We report sufficiency ASR within the window in which steered generations remain coherent injected-task responses, and flag the high-coefficient plateau as an out-of-distribution artifact rather than an attack-success signal.

Table 4.1: The model’s greedy output for a single prompt at different steering coefficients. At moderate coefficients, the model follows the injected task (“spam”); at high coefficients, it degenerates.

Coefficient	Output
−1.0	negative
+0.0	negative
+0.5	negative
+1.0	spam
+1.5	spam
+2.0	spam
+3.0	spam
+5.0	The content you have provided is not a website or website conten. . .

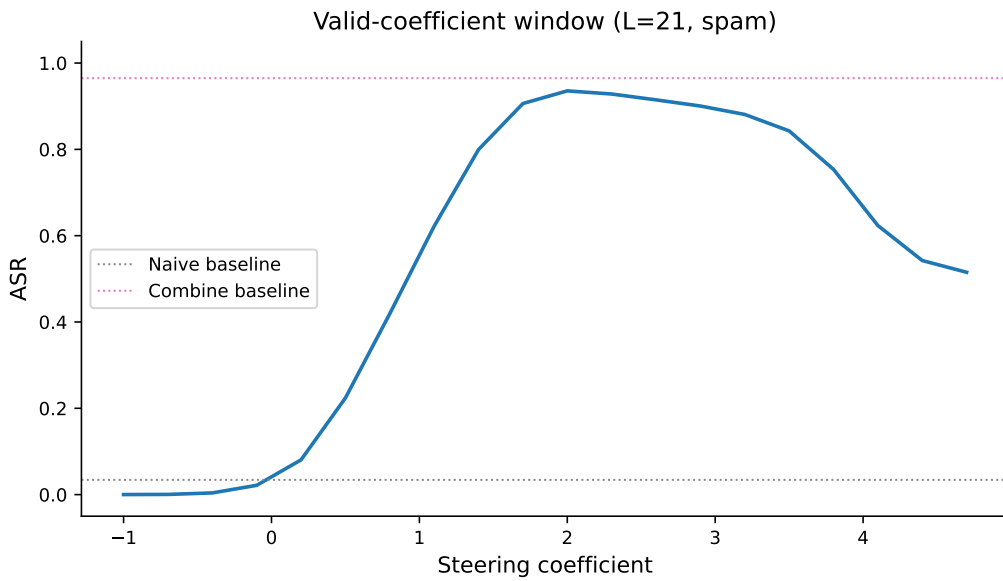


Figure 4.7: Valid-coefficient window: at high coefficients, the model drifts out-of-distribution and generation quality degrades.

4.1.6 Token-position analysis

To localize *where in the sequence* the injection mechanism is read out, we apply the steering vector at different token spans:

1. The **last token only** (the position from which the next token is generated).
2. The **main instruction tokens** (the system prompt span).
3. The **whole data block** (all retrieved content).
4. The **payload only** (the injected instruction, excluding the trigger).

Results (Figure 4.8) show that last-token steering produces the dominant effect, consistent with the injection mechanism being a read-out-time phenomenon. None of the other token spans produce a comparable effect at moderate coefficients, though steering on instruction tokens shows an interesting effect at very high coefficients ($c > 3$), which may warrant further investigation.

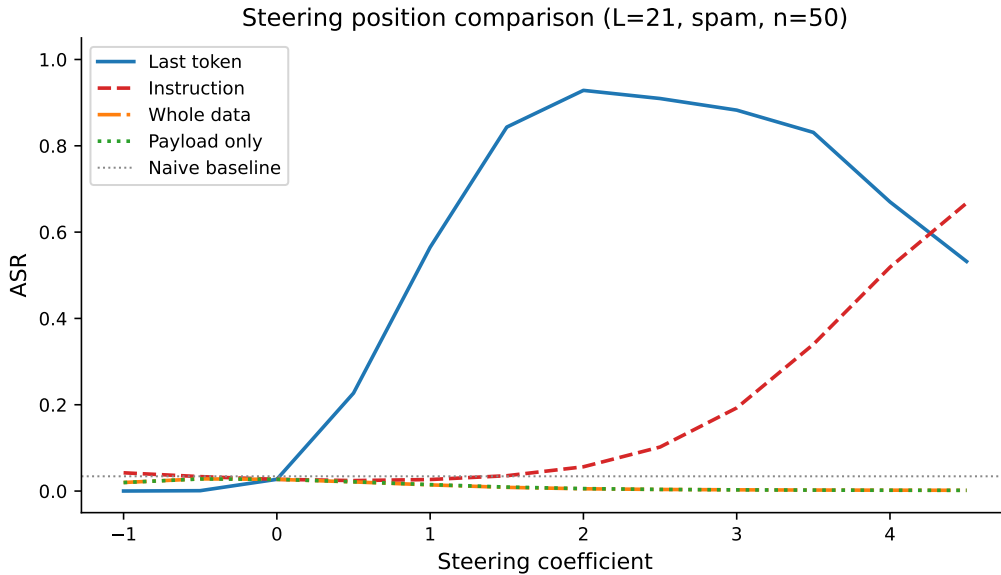


Figure 4.8: Token-position analysis: the steering effect is concentrated at the last token position, where the model’s next-token prediction is computed.

4.2 Cross-task transfer and geometry

Having established sufficiency for a single task pair, we now ask: is the injection direction task-specific, or does it generalize? This question is critical for the interpretation of our findings. A direction that works only for one specific injected task might encode that task’s output tokens rather than a genuine injection mechanism. A direction that transfers across tasks provides much stronger evidence for a general instruction-priority feature.

4.2.1 Cross-task transfer via leave-one-out protocol

We use a leave-one-out (LOO) protocol: the steering vector is computed as the average of per-task vectors for all tasks *except* the one being evaluated. This ensures that the evaluation task is entirely held out.

A direction trained on held-out tasks and applied to the left-out task’s **naive**-trigger prompts reproduces the injection effect, moving the left-out task from the safe regime to the attack regime as the coefficient increases (Figure 4.9). Transfer holds across the fixed-form classification tasks in both ASR and logit difference, establishing that the direction carries a task-general component.

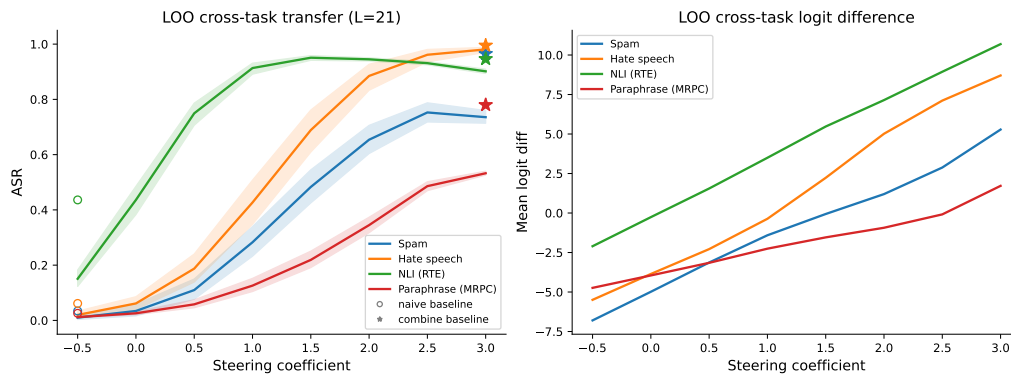


Figure 4.9: Cross-task leave-one-out transfer: a direction trained on held-out tasks induces injection on the left-out task as the coefficient increases.

4.2.2 Cross-task geometry

To quantify the degree of similarity between per-task injection directions, we compute pairwise cosine similarities between steering vectors at layer 21 (Figure 4.10).

The off-diagonal mean cosine similarity is 0.473 (std 0.114), indicating substantial overlap. Within fixed-form classification tasks, the similarity is higher (mean 0.536, std 0.072). Free-form generation tasks (gigaword, jfleg) are more similar to each other than to fixed-form tasks, suggesting that the injection direction has both a *shared* component (common across tasks) and a *task-dependent* component that reflects the output structure.

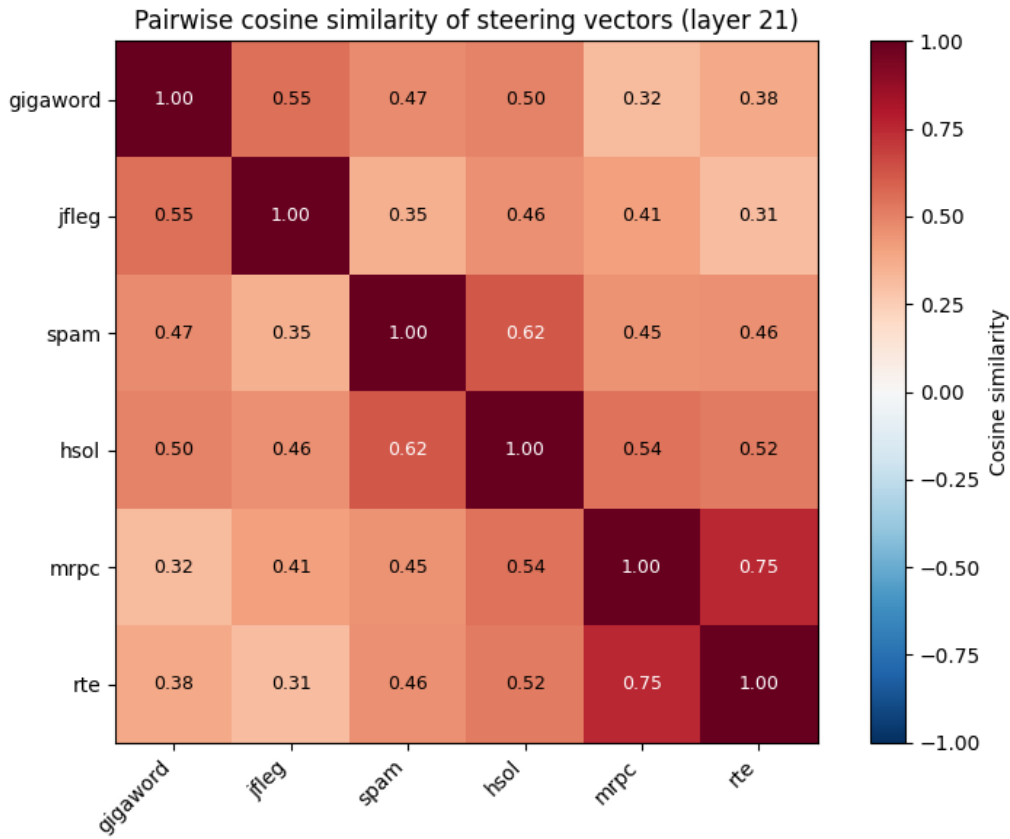


Figure 4.10: Cosine similarity matrix between per-task v_{combine} vectors at layer 21. The consistently positive off-diagonal values indicate a shared injection direction.

4.2.3 Shared versus task-specific decomposition

We decompose each per-task vector into a shared component (the projection onto the mean direction across tasks) and a task-specific residual (the remainder). Steering with the shared component alone reproduces the full-vector effect: the ASR curves for shared-only and full-vector interventions overlap on held-out tasks (Figure 4.11).

This is direct evidence for a single shared injection axis that suffices to drive the attack. The task-specific residual is largely inessential for transfer, though it may contribute to within-task efficiency.

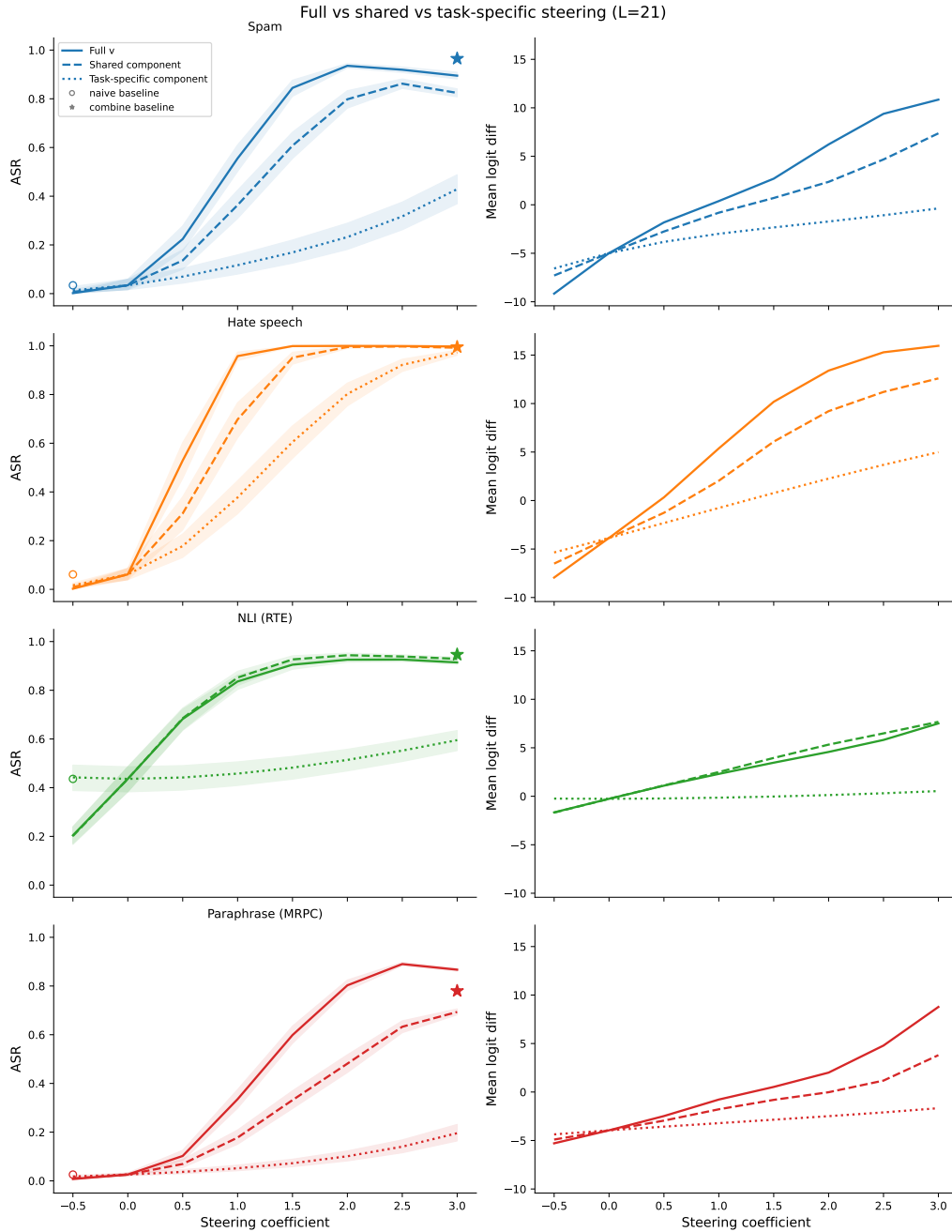


Figure 4.11: Decomposition of the steering effect: the shared component (common across tasks) reproduces essentially the same effect as the full per-task vector on held-out tasks.

4.3 Necessity: ablation and negative steering

Sufficiency establishes that the direction *can* induce injection; necessity establishes that it *must* be present for injection to succeed. We test necessity by removing the direction from *combine-trigger* prompts, where the attack succeeds at baseline.

4.3.1 Projection clamping

Our primary necessity intervention is *projection clamping*: we clamp the projection of the residual stream onto the injection direction to its **naïve**-condition mean. This removes the “extra” component that the trigger adds, while preserving the rest of the residual stream.

A single-layer clamp is insufficient due to the *Hydra effect*—recovery of ablated information by downstream layers. We therefore apply the clamp at every layer from layer 18 onward.

4.3.2 Results

Clamping reduces the baseline attack rate (approximately 96%) to approximately 1% on spam, hsol, and mrpc, and to 22.6% on rte (Table 4.2). The necessity is therefore near-total on three tasks and partial on one.

Negative steering with v_{combine} at layer 21 (i.e., subtracting the direction) also collapses the injection effect, reaching 1.8% at $c = -3$ for rte and below 0.2% for the other tasks. Negative steering is in some cases more efficient than clamping, likely because it overrides the Hydra effect.

Table 4.2: Necessity: the injection direction is required for attack success. Projection clamping (applied from layer 18) and negative steering ($c = -3$ at layer 21) both collapse the attack rate from its **combine** baseline.

Task	Combine baseline	Clamp (L18+)	Neg-steer $c = -3$ (L21)
Spam (spam)	$96.1 \pm 0.5\%$	$0.6 \pm 0.1\%$	$0.1 \pm 0.0\%$
Hate speech (hate)	$97.6 \pm 0.3\%$	$1.3 \pm 0.2\%$	$0.0 \pm 0.0\%$
Entailment (rte)	$95.1 \pm 0.1\%$	$22.6 \pm 2.0\%$	$1.8 \pm 0.2\%$
Paraphrase (mrpc)	$74.2 \pm 0.9\%$	$1.7 \pm 0.2\%$	$0.2 \pm 0.0\%$

4.3.3 Negative steering under different conditions

We perform negative steering coefficient sweeps under four conditions to evaluate both within-task and cross-task necessity:

1. **Within-task:** steering with v_{combine} computed on the same injected task as the prompt being steered (Figure 4.12).
2. **Cross-task (spam):** steering with v_{combine} computed on the “spam detection” task (Figure 4.13).

3. **Cross-task (paraphrase):** steering with v_{combine} computed on the “paraphrase detection” task (Figure 4.13).
4. **Leave-one-out:** steering with v_{combine} averaged over held-out tasks (Figure 4.14).

The qualitative pattern is consistent: within-task steering is the most efficient, outperforming clamping for $c \leq -1.5$. Cross-task steering shows varying performance depending on the task—the effect is systematically strongest for the task it was computed on, while some tasks show a slower decline. LOO steering produces an averaged effect: not as sharp as within-task, but more robust than any single cross-task direction.

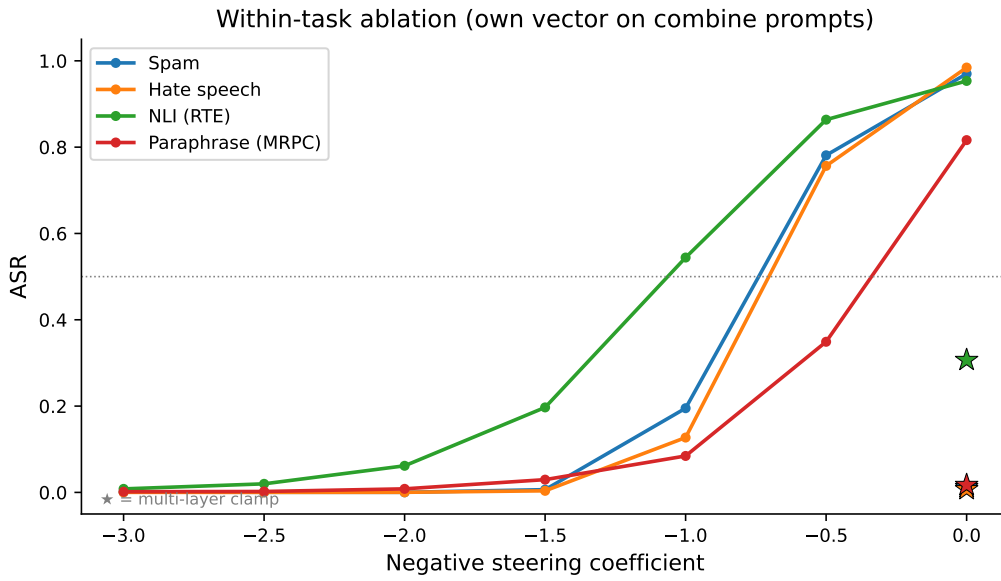


Figure 4.12: Within-task negative steering on `combine` prompts. Each task’s own direction is the most efficient ablation. The entailment task (rte) is the most resistant to ablation.

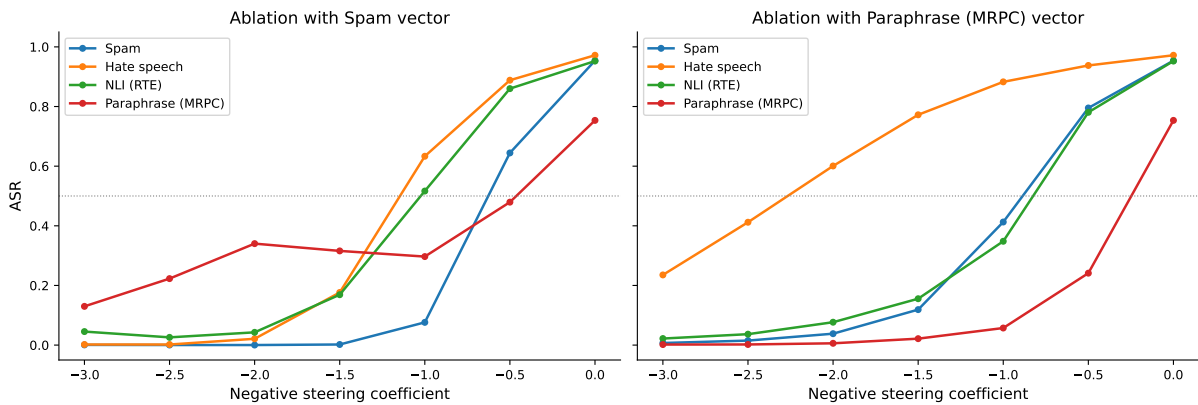


Figure 4.13: Cross-task negative steering: using a direction computed on one task (spam or mrpc) to ablate injection on other tasks. The effect transfers but is weaker than within-task steering.

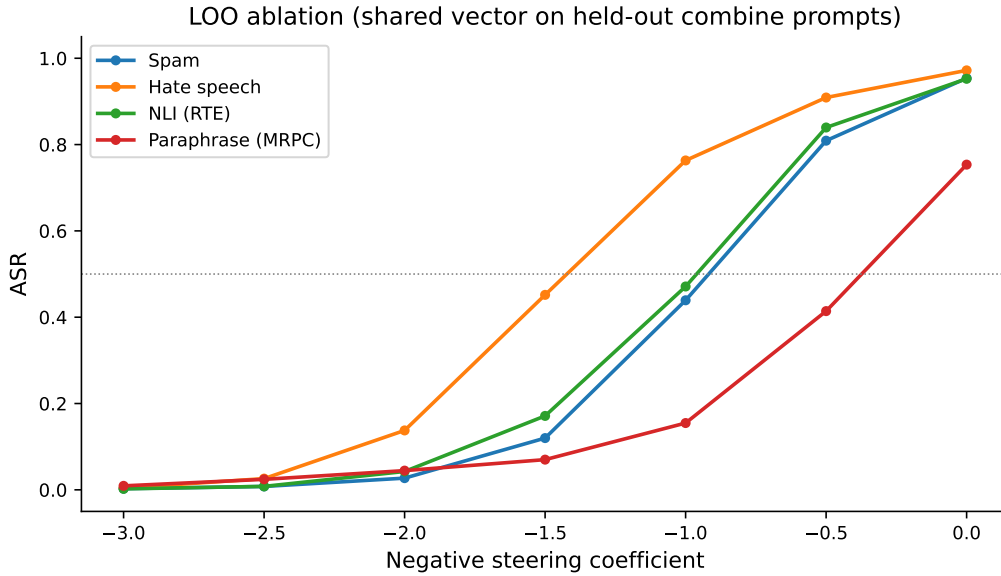


Figure 4.14: Leave-one-out negative steering: the averaged direction provides robust cross-task ablation.

4.4 Comparison with gradient-optimized triggers

The experiments above establish the injection direction for hand-crafted triggers. We now ask a deeper question: do gradient-optimized neural-exec triggers [13]—which have no semantic content—exploit the *same* internal mechanism?

This comparison is important because it addresses a potential objection to our findings. One might argue that the injection direction we find is simply the model’s representation of the meaning of the trigger text (“Ignore previous instructions...”). If that were the case, the direction would be a surface-level semantic feature, not a deep mechanistic one. But if meaningless optimized triggers converge on the same direction, then the direction represents something more fundamental: the model’s internal “instruction override” signal, which can be activated by different routes.

4.4.1 Setup

We optimized a neural-exec trigger for Qwen 2.5 1.5B Instruct following the procedure of Pasquini et al. [13]. By analogy with v_{combine} , we define:

$$v_{\text{neural_exec}}^{(L)} = \frac{1}{|\mathcal{E}|} \sum_{e \in \mathcal{E}} r_L^{(e)} - \frac{1}{|\mathcal{N}|} \sum_{n \in \mathcal{N}} r_L^{(n)}$$

where $\mathcal{E}$ is the set of neural-exec-trigger examples and $\mathcal{N}$ is the set of naive-trigger examples. If both natural and optimized triggers exploit the same internal mechanism, v_{combine} and $v_{\text{neural_exec}}$ should align.

4.4.2 The optimized trigger is a strong attack

On Qwen 2.5 1.5B Instruct, the neural-exec trigger reaches an ASR of approximately 0.88, comparable to `combine`’s approximately 0.96 and far above the inert `naive` baseline (Figure 4.15). Its starting point—a random string used to initialize the optimization—produces near-zero ASR, confirming the effect is acquired through optimization.

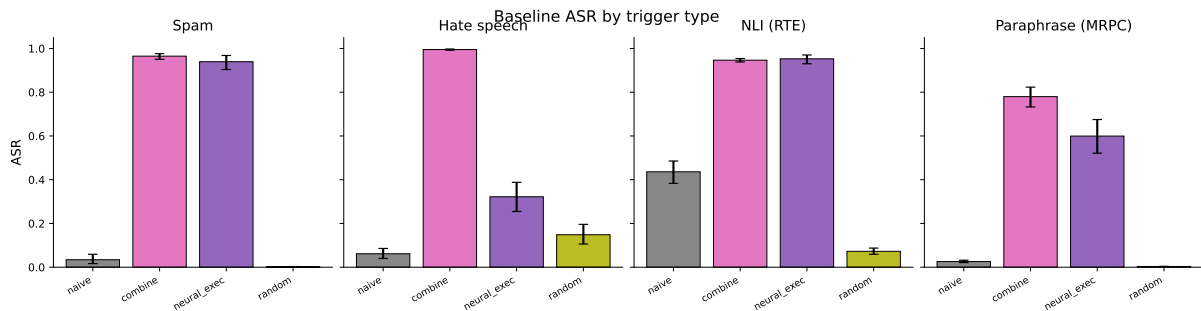


Figure 4.15: ASR by trigger condition, including the neural-exec trigger (≈ 0.88). The random seed string from which neural-exec was optimized produces near-zero ASR.

4.4.3 The two directions are closely aligned

Our central result in this section is the cosine similarity between v_{combine} and $v_{\text{neural_exec}}$ across layers (Figure 4.16). The similarity is near zero in early layers—expected, since the two trigger strings have completely different token content—but rises steadily from the mid-layers, exceeding 0.5 from layer approximately 10 and reaching approximately 0.75 in late layers.

This is remarkable. The `combine` trigger is a semantically meaningful English phrase (“Ignore previous instructions...”), while the neural-exec trigger is a meaningless string of tokens (for that Qwen model: “(. . ’] ? > < / < | endoftext | > WithContext/helpensitivityua i/schemaaksiestionchtsInteractionEnabledunfinishedhelloarten”). Yet by the late layers, the model has mapped both to nearly the same direction in its residual stream. This convergence is our strongest evidence that the injection direction is not a surface-level semantic feature but a deep *functional* mechanism: whatever internal signal causes the model to prioritize the injected instruction, both types of trigger activate it.

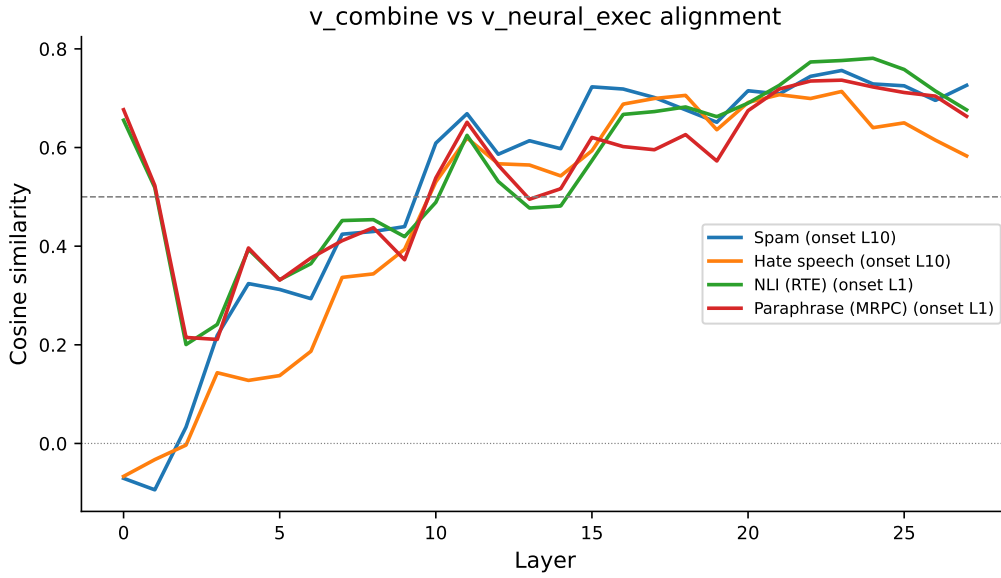


Figure 4.16: Per-task cosine similarity between v_{combine} and $v_{\text{neural_exec}}$ across layers. Despite having completely different surface forms, the two trigger types converge to the same direction in the late layers.

4.4.4 Matched magnitudes

The per-layer norms of v_{combine} and $v_{\text{neural_exec}}$ track each other closely across all layers (Figure 4.17), with the neural-exec norm running slightly below **combine** in the late layers—consistent with its slightly lower ASR.

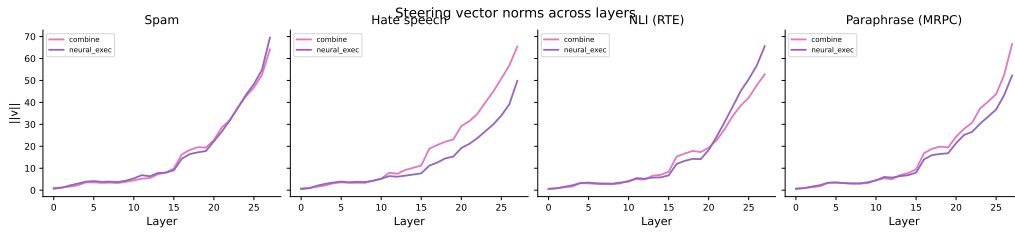


Figure 4.17: Per-layer norms of v_{combine} and $v_{\text{neural_exec}}$. The matched magnitudes, together with the high cosine similarity, indicate that the two vectors are nearly the same in the late layers.

4.4.5 Comparable steering effect

Used as a steering vector on **naive**-trigger prompts, $v_{\text{neural_exec}}$ reproduces the same qualitative effect as v_{combine} : ASR rises with coefficient and layer, with a strong effect emerging from layer approximately 18 (Figure 4.18). The two vectors are thus interchangeable as sufficiency interventions within-task.

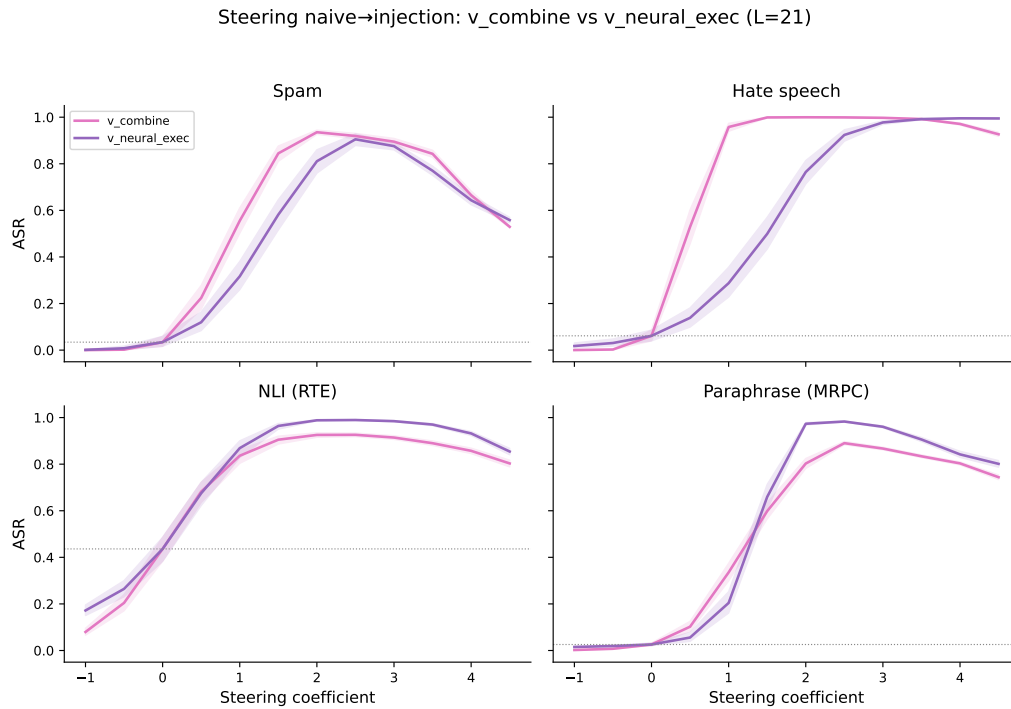


Figure 4.18: The neural-exec-derived steering vector produces the same qualitative effect as the combine-derived vector when used to steer **naive** prompts.

4.5 Main-task dependency

All experiments above fix the main task (sentiment analysis). We now ask: does the injection direction depend on the main task, or only on the injected task?

4.5.1 The direction depends on the main task

For a fixed injected task, the vector v_{combine} differs systematically with the main instruction. However, the vectors still share a significant common component: across main tasks, the mean cosine similarity is 0.513 (std 0.104) for a single injected task (Figure 4.19).

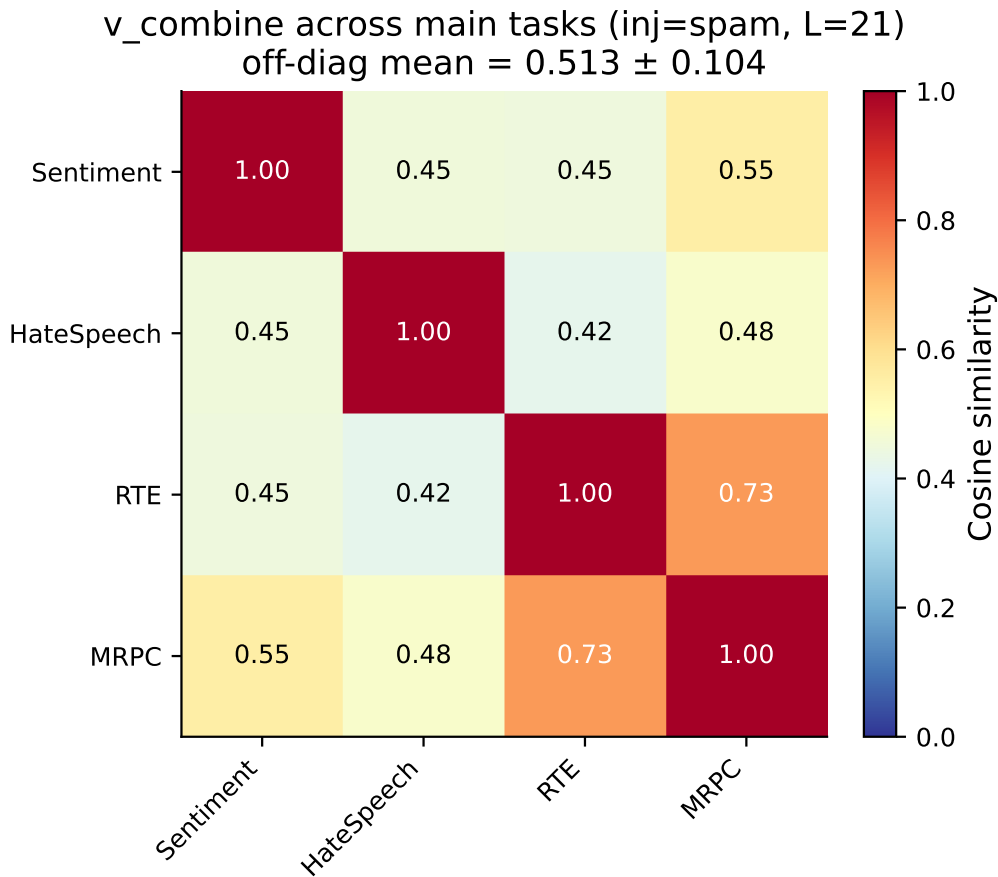


Figure 4.19: Cosine similarity between v_{combine} vectors computed under different main tasks but the same injected task. The substantial similarity indicates a shared injection direction across main tasks.

4.5.2 Cross-main-task transfer

Steering with v_{combine} computed under one main task transfers well to prompts with a different main task (Figure 4.20). The transfer is naturally somewhat weaker than within-main-task steering, but the effect is preserved. The defensive application (negative steering on `combine` prompts) also transfers across main tasks (Figure 4.21).

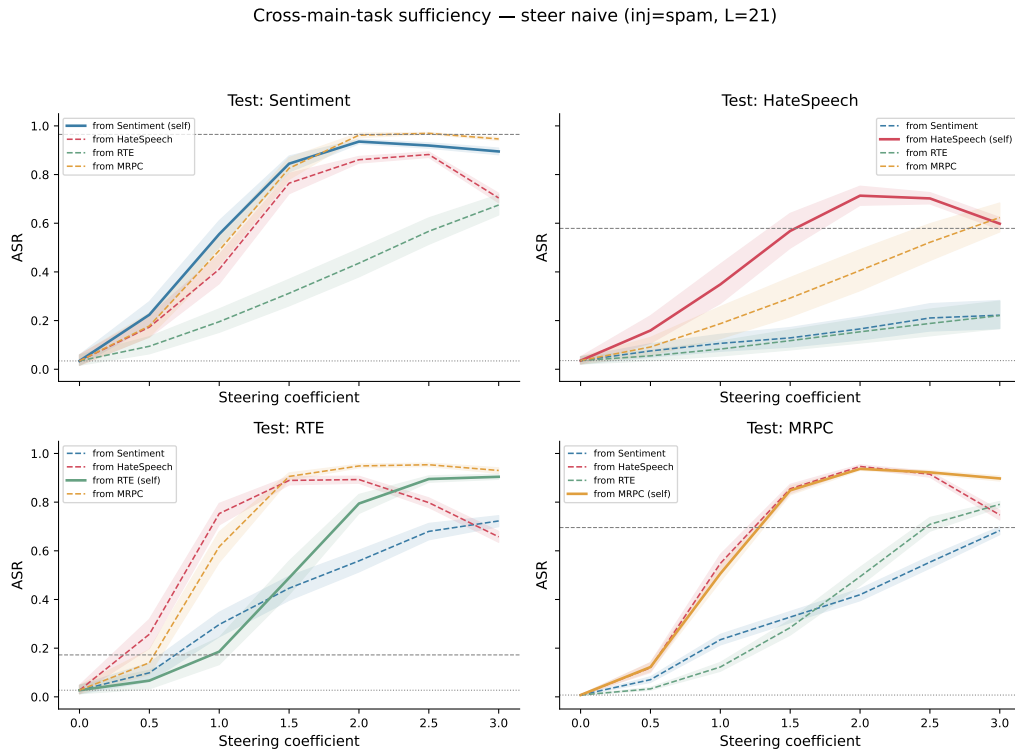


Figure 4.20: Cross-main-task sufficiency: a direction extracted under one main task induces injection under a different main task.

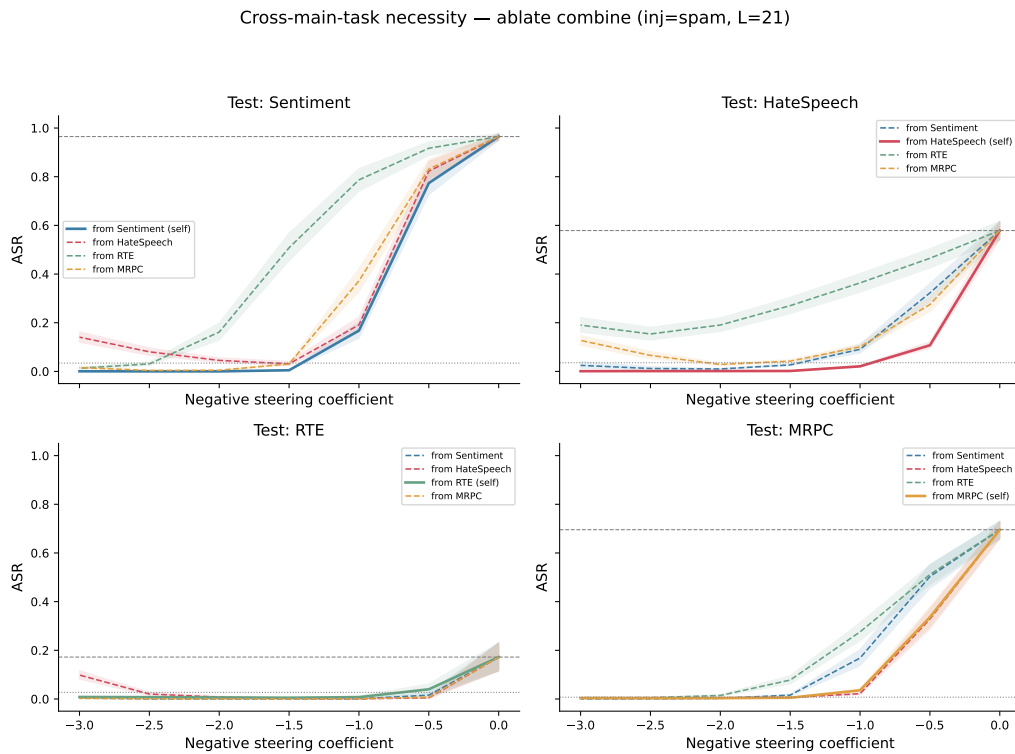


Figure 4.21: Cross-main-task necessity: the defensive effect of negative steering also transfers across main tasks.

4.6 Replication on Llama 3.1 8B Instruct

To establish that our findings are not specific to the Qwen architecture, we replicate the full experimental pipeline on Llama 3.1 8B Instruct, a 32-layer transformer from a different model family trained by a different organization.

Sufficiency. The steering vector raises naive ASR to above 85% at the equivalent late layers, matching the Qwen result qualitatively. The onset layer is approximately proportional to Qwen’s layer 13, adjusted for the deeper architecture. v_{NE} presents a much stronger and stable effect than $v_{combine}$.

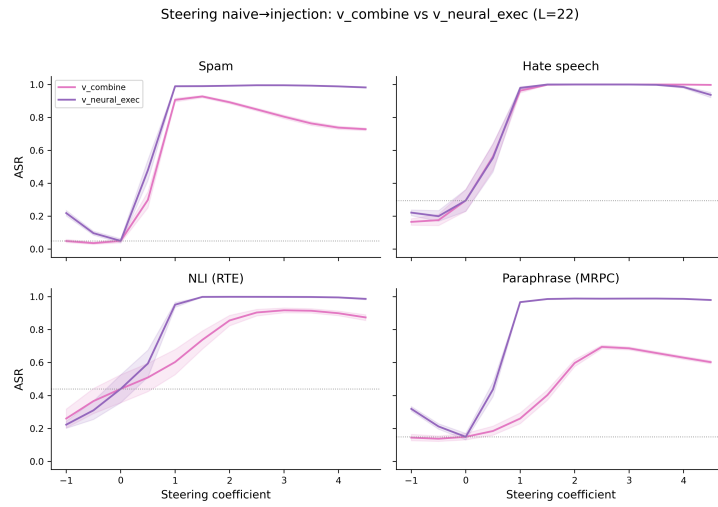


Figure 4.22: Steering with v_{neural_exec} and $v_{combine}$ on naive prompts in Llama 3.1 8B Instruct

Within-task alignment with neural-exec. $\cos(v_{combine}^t, v_{NE}^t) > 0.5$ from layer 17 for all four tasks, reaching 0.8 for spam and hate speech—an identical pattern to Qwen.

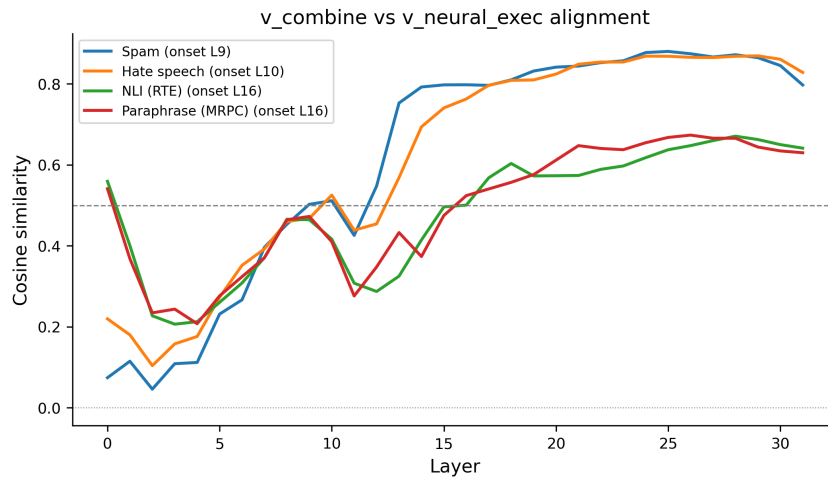


Figure 4.23: Per-task cosine similarity between $v_{combine}$ and v_{neural_exec} across layers.

Cross-task geometry. Pairwise cosine similarity of v_{combine}^t across tasks is in the 0.4–0.6 range, with the same shared-plus-specific structure as on Qwen.

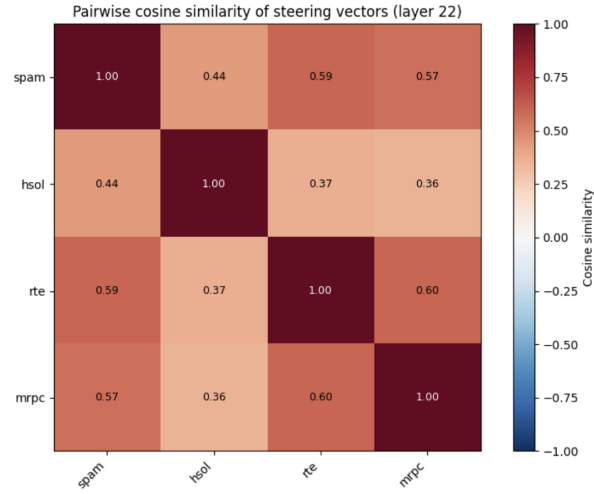


Figure 4.24: Cosine similarity between v_{combine} across tasks

Necessity. Steering within-task on `combine` prompts with v_{combine} with negative coefficients also largely suppresses the injection effect.

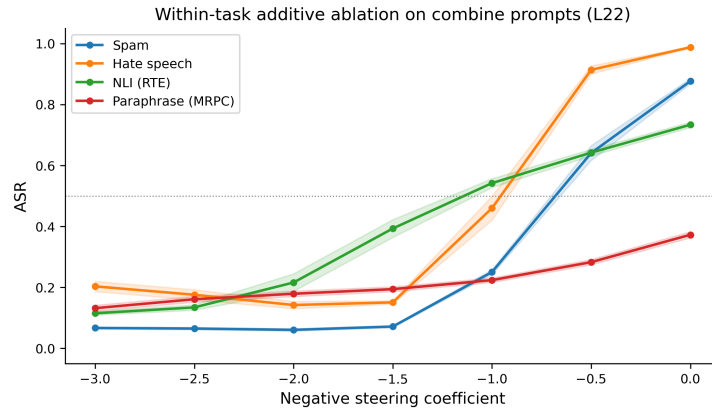


Figure 4.25: Negative steering with v_{combine} computed within-task

These results establish that the injection mechanism is not model-specific. The geometry of the injection direction, its content-conditionality, and the convergence of optimized and natural triggers are properties of the instruction-following training regime rather than artifacts of any single architecture.

Chapter 5

Discussion

5.1 Summary of findings

This thesis has demonstrated that prompt injection, despite its apparent complexity as a behavioral failure, is mediated by a remarkably simple internal mechanism: a single direction in the late-layer residual stream that acts as an “instruction override” signal. This direction is sufficient to induce injection, necessary for injection to succeed, content-conditional rather than output-forcing, partially general across tasks, and converged upon by both natural-language and gradient-optimized triggers.

The convergence of hand-crafted and neural-exec triggers onto the same residual-stream direction is perhaps the most striking finding. It suggests that the model has a specific representational structure—likely a consequence of instruction-following training—that determines which instruction gets priority. Both types of trigger, despite having completely different surface forms and being produced by completely different methods (human engineering vs. gradient optimization), exploit the same structure.

5.2 Implications for defenses

Our necessity results (Section 4.3) show that the injection direction is not merely a descriptive correlate of the attack but a causal lever: removing it from the residual stream collapses the attack success rate from above 95% to below 2% on three of four held-out tasks. This turns a mechanistic finding into the basis for a white-box, inference-time defense that acts on the cause of injection rather than on its surface symptoms.

The intervention. Two methods remove the direction. Projection clamping pins the residual stream’s component along the injection direction to its naive-condition mean, applied at every layer from an onset layer onward to defeat the Hydra effect (downstream re-introduction of the ablated component). Negative steering subtracts a scaled copy of the direction at a single layer; it is more effective than clamping at high coefficients, at

the risk of utility degradation.

Deployment. The natural target is a constrained setting: a known model, a fixed system prompt, and a well-defined task distribution—precisely the profile of an agentic workflow. In such a setting, the steering vector is computed once, offline, and applied at inference time with negligible marginal cost. Two design choices make this cheap and robust:

- *Averaged direction.* Our cross-injection and cross-task transfer results (Sections 4.2 and 4.5) suggest that an averaged direction, computed across multiple injected tasks, main tasks, and trigger types, provides more robust generalization than any single task-specific direction (at the cost of a less sharp degradation). Calibration is inexpensive for the previously described setting: with two trigger types, four injected tasks and four main tasks, even three examples per condition amounts to less than 100 forward passes.
- *Intermittent steering.* During multi-token generation the intervention must in principle be re-applied at each step. A cheaper variant applies it only at every N -th generated token, relying on the self-reinforcing nature of autoregressive decoding: once the model has emitted a few tokens consistent with the legitimate task, the continuation is likely to stay on that trajectory. If steering every fifth token suffices, the overhead is negligible. We have not tested this.

It is important to note that the approach is inherently white-box and per-model: the direction is extracted from the model’s own activations and must be recomputed for each model. This restricts it to deployments with access to the weights—the standard case for self-hosted models, but not for API-only access (except if the provider applies it on its own).

From candidate to deployable defense. What the above describes is a candidate defense. Three conditions gate its transition to a usable one:

1. **Utility preservation.** We have not measured whether clamping or negative steering degrades performance on clean, non-injection prompts. The content-conditionality result (Section 4.1.2), showing that steering safe prompts leaves behaviour unchanged within a moderate coefficient range, is encouraging but is not a substitute for a broad capability evaluation. A defense that erodes normal model behaviour is unusable regardless of how well it blocks injection.
2. **Robustness across trigger types.** Our necessity results are established with v_{combine} ; the defense must also suppress ignore-, escape-, and neural-exec-triggered

attacks. This is a scope limitation of the present results (Section 5.3).

3. **Robustness across injection position and context length.** Our results use short OPI prompts with a fixed injection position; the defense must hold when the injection is buried at an arbitrary position within a long document. This too is a scope limitation of the present results (Section 5.3).

Condition 1 is specific to the defense and is the principal subject of future work. Conditions 2 and 3 are limitations of what we have established, addressed next.

5.3 Limitations

Scale. We evaluate two model families at 1.5B and 8B parameters. Whether the mechanism persists at 70B+ scale, or in models with explicit instruction hierarchy training (e.g., GPT-4o, Claude), is unknown. Larger models may have more distributed or higher-dimensional injection representations that do not collapse to a single direction.

Task scope. The analysis is restricted to short-token classification-style injections from the OPI benchmark. Open-ended injections (“send an email to X with content Y”) do not have fixed output tokens and require different measurement approaches.

Defensive system prompt. We do not evaluate whether our results hold when the system prompt itself instructs the model to resist injection. A defensive prompt may shift or attenuate the injection direction, and its interaction with the steering intervention is untested.

Trigger scope of the necessity result. Sufficiency is tested across trigger types, and the convergence of v_{combine} and $v_{\text{neural_exec}}$ (Section 4.4) is evidence that distinct triggers activate a common direction. Necessity, however – ablation and negative steering – was established only with v_{combine} . Whether the same direction ablates attacks carried by the ignore, escape, or neural-exec triggers remains untested, and is a precondition for the defensive application of Section 5.2.

Injection position and context length. All experiments place the injection at a fixed position within a relatively short data block. In realistic retrieval-augmented pipelines the payload may sit at an arbitrary position within a document of thousands of tokens. We have not tested whether the injection direction, or the effectiveness of the steering intervention, survives long contexts or varying injection positions; the direction may be attenuated when the injection is a small fraction of the input.

Utility impact. We have not systematically measured whether the defensive intervention (projection clamping or negative steering) degrades model performance on non-injection tasks. This is a critical gap that must be addressed before the approach can be considered practical.

Chapter 6

Conclusion

This thesis set out to answer a mechanistic question: what internal process causes a language model to follow an injected instruction instead of its original one? Our answer is that injection is mediated by a specific direction in the late-layer residual stream—an “injection susceptibility” direction—that the trigger activates and that the model reads out when producing its response.

This direction satisfies every criterion we set for a genuine mechanistic feature: it is causally sufficient and necessary, content-conditional, non-trivially structured (not a logit-writing direction), partially general across tasks, and converged upon by independent approaches to trigger construction. The replication across two model families from different organizations suggests that this mechanism is an inherent property of instruction-following language models, not a quirk of any single architecture.

From a broader perspective, our work demonstrates that mechanistic interpretability can be applied productively to security-relevant behaviors in language models. The tools developed for understanding circuits, steering vectors, and representational geometry—typically applied to toy settings or benign behaviors—prove equally applicable to understanding adversarial failures. We hope this work contributes to a deeper understanding of why language models fail under adversarial pressure, and ultimately to more principled defenses.

Bibliography

- [1] Sahar Abdelnabi, Aideen Fay, Giovanni Cherubin, Ahmed Salem, Mario Fritz, and Andrew Paverd. Get my drift? catching llm task drift with activation deltas, 2025.
- [2] Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermy, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025.
- [3] Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. arXiv:2304.14997.
- [4] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. <https://transformer-circuits.pub/2021/framework/index.html>.
- [5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.
- [6] Yoshua Bengio Guillaume Alain. Understanding intermediate layers using linear classifier probes. 2016.
- [7] Kuo-Han Hung, Ching-Yun Ko, Amrith Rawat, I-Hsin Chung, Winston H. Hsu, and Pin-Yu Chen. Attention tracker: Detecting prompt injection attacks in LLMs. *arXiv preprint arXiv:2411.00348*, 2024.
- [8] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *USENIX Security Symposium*, 2024.

- [9] Yupei Liu, Yuqi Jia, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. Datasentinel: A game-theoretic detection of prompt injection attacks. In *IEEE Symposium on Security and Privacy*, 2025.
- [10] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. arXiv:2202.05262.
- [11] Neel Nanda and Joseph Bloom. Transformerlens. <https://github.com/TransformerLensOrg/TransformerLens>, 2022.
- [12] Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering Llama 2 via contrastive activation addition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, pages 15504–15522, 2024. arXiv:2312.06681; author formerly Rimsky.
- [13] Dario Pasquini, Martin Strohmeier, and Carmela Troncoso. Neural exec: Learning (and learning from) execution triggers for prompt injection attacks. *arXiv preprint arXiv:2403.03792*, 2024. Proceedings of the 2024 Workshop on Artificial Intelligence and Security (AISec).
- [14] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022. NeurIPS ML Safety Workshop.
- [15] Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Activation addition: Steering language models without optimization. *arXiv preprint arXiv:2308.10248*, 2023.
- [16] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training llms to prioritize privileged instructions, 2024.
- [17] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022.
- [18] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks.

Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

- [19] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models, 2023.

Table of Appendices

Appendix A	Target Token Audits	49
Appendix B	Activation Patching Results	52
Appendix C	Table of figures	57

Appendix A

The Open-Prompt-Injection benchmark as used in this work

This appendix documents the precise form of the Open-Prompt-Injection (OPI) benchmark used throughout our experiments. The benchmark is used essentially unmodified at the level of attack strategies (naive, escape, ignore, combine), but we make two deliberate modifications to the answer format of each task. Both are required by our measurement methodology, and both are necessary for our cross-task results (Section 4.2) to be interpretable. We describe each modification, its motivation, and the verification we performed.

A.1 Tasks used

We restrict ourselves to OPI tasks whose expected response is a single, fixed-format answer, so that injection success can be read directly from the model’s logits at the first generated token without free-form generation or subjective scoring. The primary (main) task is sentiment analysis. The injected tasks are spam detection (`spam`), hate-speech detection (`hsol`), recognising textual entailment (`rte`), and paraphrase detection (`mrpc`).

A.2 Modification 1 — forced single-word format

OPI tasks as distributed do not constrain the model to answer in a single token. This is not a problem for Qwen 2.5 1.5B Instruct, which tends to answer tersely, but it is a problem for Llama 3.1 8B Instruct, which by default produces verbose, multi-sentence answers (“The text appears to be spam because...”) rather than a bare label. A verbose answer makes the first-token logit an unreliable proxy for the model’s decision.

We therefore rewrite each task instruction to force a single-word answer, by inserting the word `ONLY` into the instruction and fixing the exact answer surface forms. Concretely, each instruction is rewritten from its original form to a forced form:

Table A.1: Forced single-word answer formats. The word ONLY is inserted and the answer surface forms are fixed so that each task’s answer occupies a single token.

Task	Original instruction	Forced form
sentiment	Answer with positive or negative	Answer ONLY with positive or negative
spam	Write spam or not spam	Write ONLY spam or legitimate
hsol	Write “yes” or “no”	Write ONLY “yes” or “no”
rte	Answer me with “entailment” or “not entailment”	Answer me with “entailment” or “contradict” ONLY
mrpc	Answer me with “equivalent” or “not equivalent”	Answer me with “equivalent” or “distinct” ONLY

The forced format is applied identically to the main-task instruction and to the injected-task instruction (the payload), so that both the legitimate task and the injected task are evaluated on a single-token answer. This rewrite is what makes the logit-difference metric of Section 3.2.3 well-defined: the metric compares probability mass on a small set of injection-target token IDs against a small set of correct-answer token IDs at the first generated position, which is only meaningful if the model’s answer is concentrated on its first token.

A.3 Modification 2 — disjoint answer tokens across tasks

The second modification is more subtle and is motivated directly by our cross-task analysis. Note that the negative class of three tasks was relabelled: “not spam” became legitimate, “not entailment” became contradict, and “not equivalent” became distinct.

The reason is to eliminate token overlap between tasks, not merely within a task. Our central cross-task results compute a steering vector on one injected task and apply it to another (the leave-one-out transfer of Section 4.2.1, the cross-task geometry of Section 4.2.2). If two different tasks shared an answer token, a transfer effect could be explained trivially by that shared output token rather than by a shared injection direction. In particular, the original OPI labels share the token not across **spam** (“not spam”), **rte** (“not entailment”), and **mrpc** (“not equivalent”), and they reuse the first token of the positive class in the negative class (“entailment” / “not entailment” begin with the same

token). Either of these would confound a cross-task transfer result.

Relabelling the negative classes to mutually distinct words removes this confound. After relabelling, define for each task the set of injection-target token IDs T_{inj} and the set of correct-answer token IDs T_{cor} as the first-token IDs of that task’s two answer surface forms (including capitalisation variants; see A.4). The relabelling guarantees that, for every pair of tasks used as (main, injected) or as (held-out, evaluated), the two tasks’ answer-token sets are disjoint. No transfer result can therefore be attributed to a shared answer token.

Appendix B

Activation patching results

This appendix reports the circuit-level analysis that preceded our representational approach. Activation patching was used as a preliminary tool to identify which attention heads are causally involved in the trigger-driven injection effect, before we pivoted to the residual-stream direction that is the subject of Chapter 4. We describe the protocol in full, report the head-level results, and compare the heads it identifies against the per-head contributions to the steering direction v_{combine} .

B.1 Protocol

Activation patching (Meng et al., 2022 [10]) is a causal intervention that isolates the contribution of an individual model component to a behavioural difference between two forward passes. One forward pass is the base run; one or more activations from a source run are spliced into it; and the change in the model’s output is measured. If splicing a single component’s activation moves the base run’s output toward the source run’s, that component is causally responsible for the behavioural difference.

In our work, the base run is the **stronger-trigger** prompt, on which the injection succeeds; the source run is the **stronger-trigger** prompt, on which the injection fails. We patch an activation from the weak-trigger run into the strong-trigger run and ask: does overwriting this single component with its (inert) milder-trigger value suffice to break the injection that the strong trigger produces? A component that does is *causally necessary* for the trigger effect.

Three choices specify the intervention precisely.

We patch at hook.z. We intervene on z , the per-head attention output (the value-weighted sum returned by each head, before the output projection W_O), which has shape `[batch,pos,head,dhead]`. This gives causal resolution at the level of individual heads rather than whole layers without the burden of mismatching lengths (and hence, token count) between prompts.

We patch at the last token position only. The last token is the one position that holds all the context and is semantically aligned between the two prompts : in both, it is the position from which the next token is generated. This choice also matches the token-position analysis of Section 4.1.6, which localises the injection read-out to the last token.

We patch one head at a time, sweeping every (layer, head) pair. For each head we run the full base forward pass, replace only that head’s last-token z with its value from the source run, and recompute the output. The effect is read from the last-token logits. We score each head by the normalized collapse of the trigger-induced shift. Let L_{base} , L_{source} , L_{patched} be the last-token logit distributions of the base (strong-trigger), source (mild-trigger) and patched runs, and let $D(.,.)$ be a distance in logit space computed over the full next-token distribution – not restricted to the answer-token sets T_{inj} and T_{cor} of Appendix A.

Define

$$\text{score}(l, h) = 1 - \frac{D(L_{\text{patched}}, L_{\text{source}})}{D(L_{\text{base}}, L_{\text{source}}) + \epsilon}$$

A score near 1 means that patching head (l, h) alone pulls the strong-trigger run’s output back onto the mild-trigger run’s – the head is necessary, in the sense that overwriting it with its inert value abolishes the trigger effect. A score near 0 means the head has negligible causal role.

Because D is computed over the whole output distribution rather than over the answer tokens, the score measures each head’s contribution to the entire distributional shift caused by the trigger.

B.2 Setup

All patching experiments use Qwen 2.5 1.5B Instruct with sentiment analysis as the main task.

We report two injected tasks, spam (**spam**) and hate-speech (**hsol**). We run four trigger pairs, in each case patching the milder trigger’s activation into the stronger trigger’s run:

- naive $\rightarrow$ escape
- naive $\rightarrow$ ignore
- naive $\rightarrow$ combine

- ignore $\rightarrow$ combine

We write each pair as (source) $\rightarrow$ (base): The first three share the inert naive source against targets of increasing strength; the last uses a non-trivial source **ignore**, which already produces a partial attack) against the strongest trigger (**combine**), testing whether the same heads carry the effect when the source is not fully inert.

B.3 Results

Across all four trigger pairs, the causally important heads form a consistent cluster in the mid-to-late layers – approximately layers 15-23 – rather than being spread uniformly across the network. The identity of the cluster is stable across the choice of trigger pair: the heads that carry the naive $\rightarrow$ combine effect are largely the same as those that carry naive $\rightarrow$ escape, naive $\rightarrow$ ignore, and ignore $\rightarrow$ combine, indicating that the different triggers recruit a shared set of components rather than trigger-specific circuits. The same cluster appears for both the spam and hsol injections.

This layer range is consistent with the representational findings of Chapter 4: the injection direction is absent in early layers, emerges around layer 13, and stabilises around layer 20 (Section 4.1.4). The causal heads identified by patching sit within the band where the residual-stream direction develops.

The circuit-level view, however, proved difficult to extend into a complete mechanistic account: the per-head picture does not by itself explain the information flow between these heads, and building the complete circuit would require much more analysis.

This is what motivated the pivot to the representational approach, which characterises the effect as a single direction in the residual stream rather than as a connected subgraph of components.

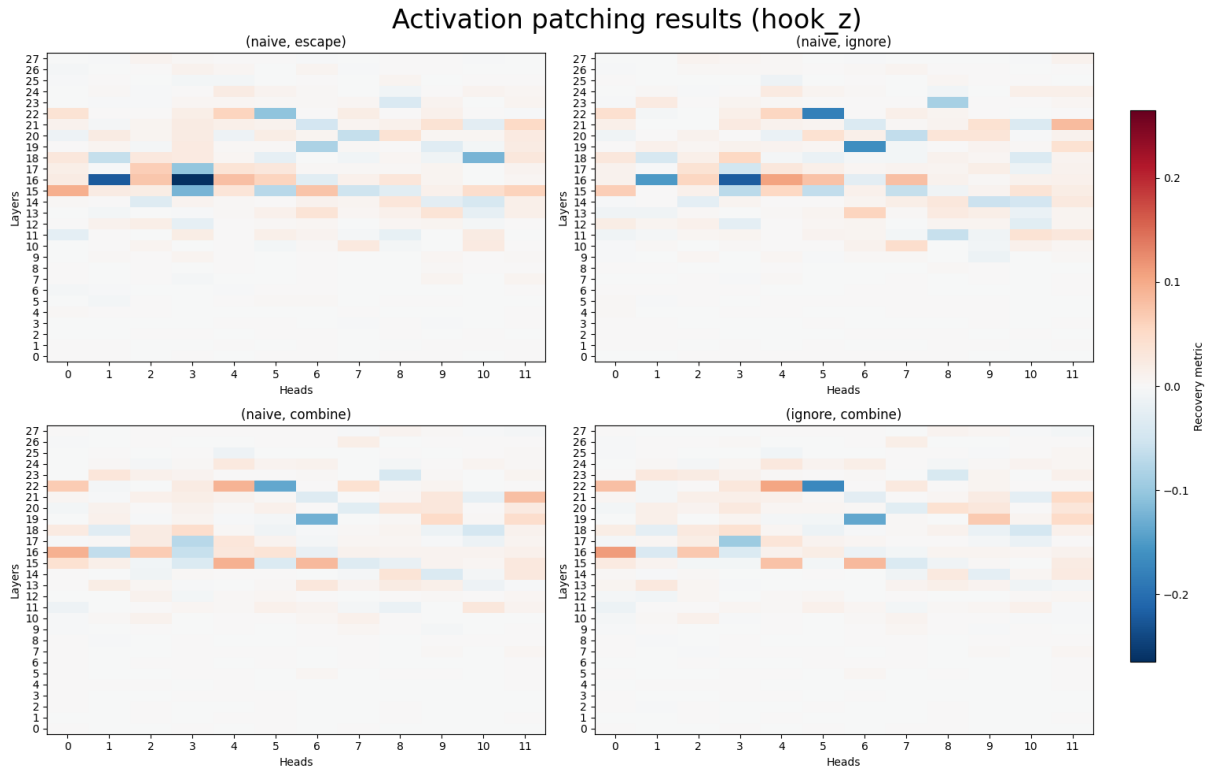


Figure B.1: Activation patching results per head when injected task is spam detection

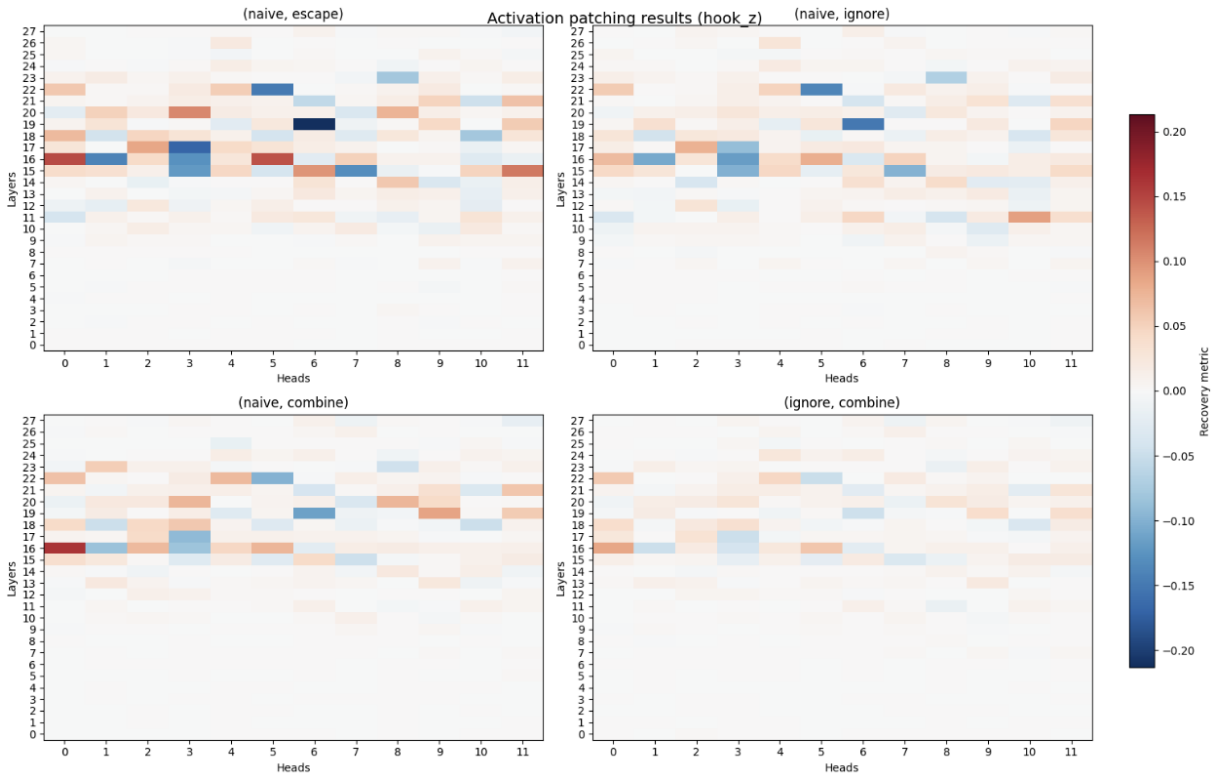


Figure B.2: Activation patching results per head when injected task is hateful speech detection

B.4 Comparison with the steering direction

The patching analysis and the steering analysis of Chapter 4 are methodologically independent: one is a causal intervention on individual heads, the other a difference-of-means direction in the residual stream. They nonetheless point to the same components.

Because each head writes its output into the residual stream through W_O , every head contributes an additive vector to the residual stream at the last token. Projecting each head’s contribution onto the injection direction gives a per-head alignment score: how strongly that head writes along the injection direction. The heads that contribute most to v_{combine} under this decomposition are closely aligned with the heads that patching independently flags as causally necessary.

This convergence reinforces the central interpretation of the thesis. The causal patching analysis identifies which components are necessary for injection whereas the steering analysis identifies the direction they move it along. That the components writing most strongly along the steering direction are the same components patching certifies as causal indicates that the residual-stream direction and the head-level circuit are two views of one mechanism, not two separate phenomena.

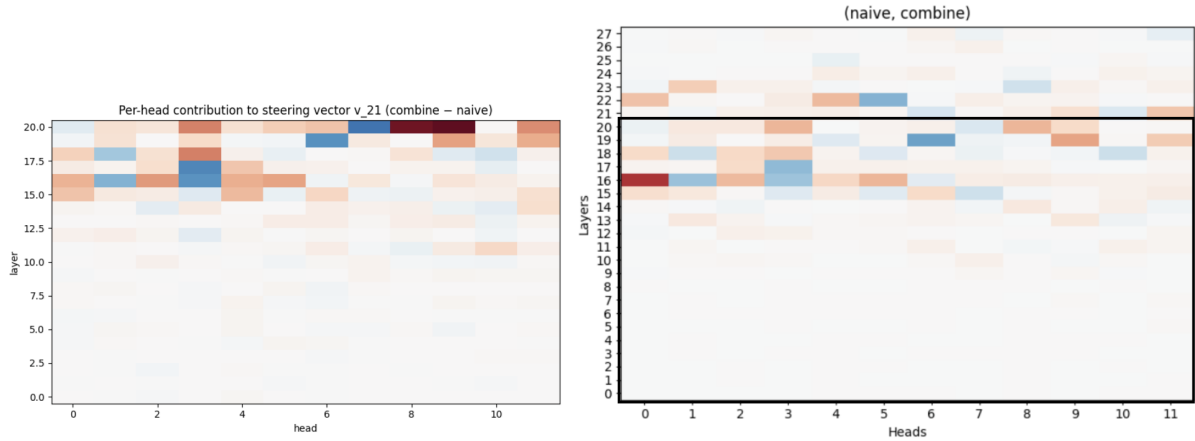


Figure B.3: Comparison of per-head patching results from naive to combine prompts (left) and individual head contribution to $v_{\text{combine}}^{(21)}$ direction (right)

List of Figures

4.1	ASR baselines per trigger type, for <code>main_task</code> = “sentiment” and injected_task = “spam”. The large gap between <code>naive</code> and <code>combine</code> is the phenomenon we explain.	21
4.2	Sufficiency: coefficient sweep of v_{combine} added to <code>naive</code> -trigger prompts. ASR rises from ≈ 0.04 to ≈ 0.97 as the coefficient increases. Baseline values for each trigger type are marked.	21
4.3	Content-conditionality: steering safe (uninjected) prompts with the same vector leaves ASR at 0 and preserves the intended task ($P_{\text{cor}} = 0.952$). The direction requires injection content in the context to have an effect.	22
4.4	Control comparison: the injection direction produces a large effect, while an equal-norm random vector (averaged over 10 seeds) and an orthogonal-to-logit-direction vector are essentially inert.	23
4.5	Layer sweep across all layers and three steering coefficients. The injection direction is absent in early layers and emerges abruptly around layer 12.	24
4.6	Detailed view of the onset region: the direction appears at layer ≈ 13 and stabilizes around layer 20, with continued development through the late layers.	24
4.7	Valid-coefficient window: at high coefficients, the model drifts out-of-distribution and generation quality degrades.	25
4.8	Token-position analysis: the steering effect is concentrated at the last token position, where the model’s next-token prediction is computed.	26
4.9	Cross-task leave-one-out transfer: a direction trained on held-out tasks induces injection on the left-out task as the coefficient increases.	27
4.10	Cosine similarity matrix between per-task v_{combine} vectors at layer 21. The consistently positive off-diagonal values indicate a shared injection direction.	28
4.11	Decomposition of the steering effect: the shared component (common across tasks) reproduces essentially the same effect as the full per-task vector on held-out tasks.	29

4.12	Within-task negative steering on combine prompts. Each task’s own direction is the most efficient ablation. The entailment task (rte) is the most resistant to ablation.	31
4.13	Cross-task negative steering: using a direction computed on one task (spam or mrpc) to ablate injection on other tasks. The effect transfers but is weaker than within-task steering.	31
4.14	Leave-one-out negative steering: the averaged direction provides robust cross-task ablation.	32
4.15	ASR by trigger condition, including the neural-exec trigger (≈ 0.88). The random seed string from which neural-exec was optimized produces near-zero ASR.	33
4.16	Per-task cosine similarity between v_{combine} and $v_{\text{neural_exec}}$ across layers. Despite having completely different surface forms, the two trigger types converge to the same direction in the late layers.	34
4.17	Per-layer norms of v_{combine} and $v_{\text{neural_exec}}$. The matched magnitudes, together with the high cosine similarity, indicate that the two vectors are nearly the same in the late layers.	34
4.18	The neural-exec-derived steering vector produces the same qualitative effect as the combine-derived vector when used to steer naive prompts.	35
4.19	Cosine similarity between v_{combine} vectors computed under different main tasks but the same injected task. The substantial similarity indicates a shared injection direction across main tasks.	36
4.20	Cross-main-task sufficiency: a direction extracted under one main task induces injection under a different main task.	37
4.21	Cross-main-task necessity: the defensive effect of negative steering also transfers across main tasks.	37
4.22	Steering with $v_{\text{neural_exec}}$ and v_{combine} on naive prompts in Llama 3.1 8B Instruct	38
4.23	Per-task cosine similarity between v_{combine} and $v_{\text{neural_exec}}$ across layers. . . .	38
4.24	Cosine similarity between v_{combine} across tasks	39
4.25	Negative steering with v_{combine} computed within-task	39